

Timed Automata Verification and Synthesis via Finite Automata Learning

Ocan Sankur

Univ Rennes, Inria, CNRS, Irisa, Campus de Beaulieu, Rennes, 35042,
France.

Contributing authors: ocan.sankur@cnrs.fr;

Abstract

We present algorithms for model checking and controller synthesis of timed automata, seeing a timed automaton model as a parallel composition of a large finite-state machine and a relatively smaller timed automaton, and using compositional reasoning on this composition. We use automata learning algorithms to learn finite automata approximations of the timed automaton component, in order to reduce the problem at hand to finite-state model checking or to finite-state controller synthesis. We present an experimental evaluation of our approach.

Keywords: Timed automata, model checking, assume guarantee reasoning, automata learning, compositional verification

1 Introduction

Timed automata [1] are a well-known formalism for modeling and verifying real-time systems. They can be used to model systems as finite automata, while using, in addition, clocks to impose timing constraints on the transitions. Using clock variables have advantages. They allow one to describe models that are expressive thanks to real-valued clock values; moreover, the use of specific clock variables enable optimizations such as sound and complete abstractions, also known as extrapolation operators [2]. Model checking algorithms have been developed and implemented in tools such as Uppaal [3], TChecker [4], PAT [5].

One approach for model checking timed automata is based on representing the set of clock values with *zones*, which are particular polyhedra, and using explicit enumeration on the discrete states. There has been extensive research on sound and complete

abstractions on zones, which improved the performance of the model checking tools, and made it possible to handle models with more complex time constraints; see [6] for a survey. However this approach does not scale to models with large discrete spaces due to explicit enumeration. Several authors have developed algorithms to remedy this issue, and to attempt to adapt efficient model checking techniques finite-state systems to timed systems. Extensions of binary decision diagrams (BDD) with clock constraints have been considered both for continuous time [7–9] and discrete time [10, 11]. Another approach is to use predicate abstraction on clock variables that enables efficient finite-state verification techniques based on BDDs or SAT solvers [12–14].

Controller synthesis is a related problem in which some transitions of the system are controllable and some are uncontrollable, and the objective is to compute a control strategy which guarantees that all induced runs of the system satisfy a given specification; see *e.g.* [15]. This problem is formalized using *games*, and in the case of real-time systems, using timed games [16, 17]. Zone-based algorithms have been developed to solve timed games and compute control strategies [18], and are available in the Uppaal TIGA tool [19]. These algorithms suffer from the same limitations as the zone-based model checking algorithms. Although they can be efficient on instances with small discrete state spaces, they do not scale well to large systems. An attempt was made to implement the counter-example guided abstraction refinement scheme to handle larger discrete state space in timed games in [20]. On the other hand, there are several efficient finite-state game solvers, based on BDDs and SAT solvers, which can efficiently handle relatively large state spaces [21], but cannot handle real time.

In this work, we introduce an approach that is applied both to model checking and controller synthesis of timed automata with the objective of combining the advantages of both timed automata and finite-state model checkers and game solvers. Our suggestion is to see the input model, without loss of generality, as a parallel composition between a finite-state machine $\mathcal{A}$, and a timed automaton $\mathcal{T}$. We specifically target instances where $\mathcal{A}$ is large, and $\mathcal{T}$ is relatively small but nontrivial. Note that this point of view was considered before in the verification of synchronous systems within a real-time environment [22]. As a novelty, for model checking, we apply a compositional reasoning rule on the product $\mathcal{A} \parallel \mathcal{T}$ by replacing the timed automaton $\mathcal{T}$ by a (small) deterministic finite automaton (DFA) H which represents the behaviors of $\mathcal{T}$. To automatically select the DFA H , we adapt the algorithm [23] to our setting, and use a DFA learning algorithm (such as L^* [24], or TTT [25]) to find an appropriate DFA either to prove the specification or to reveal a counterexample.

Our approach enjoys the principle of *separation of concerns* in the following sense. A timed automaton model checker is used by the learning algorithm to answer *membership* and *equivalence queries* (see Section 2.3); these are answered without referring to $\mathcal{A}$, thus, by avoiding the large discrete state space. Therefore, the timed automaton model checker is used in this approach for what it is designed for: handling real-time constraints encoded in $\mathcal{T}$, not for dealing with excessive discrete state spaces. Once an appropriate DFA H is found by the learning algorithm, the system $\mathcal{A} \parallel H$ is model-checked using a finite-state model checker whose focus is to deal with large *discrete* state spaces. We can thus benefit from the best of the two worlds: a state-of-the-art model checker for

timed automata, which is somewhat used here as a theory solver, and any finite-state model checker based on BDDs, SAT solvers, or even explicit-state enumeration.

Note that this approach is limited to checking untimed properties of timed automata. This is not much of a loss compared to the literature since most model checking tools only support reachability or safety properties, while some (such as Uppaal) do offer a limited language to express timed properties. But the latter can be reduced to reachability by adding small tester component as described in [26]; so by way of modeling, it is still possible to check some timed properties with the method presented in this paper, though it is not possible to support full logics such as metric temporal logic, or timed CTL.

The application of the learning-based compositional reasoning of [23] to controller synthesis is more involved. Our objective was to find a way to exploit efficient finite-state game solvers [21] in the context of timed automata even if this meant having an incomplete algorithm. We describe a setting where a one-sided abstraction is applied for controller synthesis by replacing the timed automaton component by a learned DFA. Contrarily to the model checking algorithm, our controller synthesis algorithm is sound but not complete, that is, the algorithm may fail although there exists a control strategy, while any control strategy that is output is correct. More precisely, we consider timed games in the form $\mathcal{G} \parallel \mathcal{T}$ where $\mathcal{G}$ is a finite-state game, and $\mathcal{T}$ is a timed automaton. We describe an algorithm that alternates between two phases. In the first phase, the goal is to find a DFA $\overline{H}$ that is an overapproximation of $\mathcal{T}$. Once this is found, we use a finite-state game solver on $\mathcal{G} \parallel \overline{H}$; if there is a control strategy, we prove that it can be applied in the original system $\mathcal{G} \parallel \mathcal{T}$. If not, then we obtain a counterstrategy $\mathfrak{S}$. We then switch to the second phase whose goal is to check whether the counterstrategy is spurious or not; and it does so by learning an underapproximation DFA $\underline{H}$ of $\mathcal{T}$, and checking whether $\mathfrak{S}$ induces runs that are all in $\underline{H}$. Accordingly, we either reject the instance or switch back to the first phase. As in the model checking algorithm, the timed automaton model checker is only used to answer queries independently from $\mathcal{G}$, and a finite-state game solver and a model checker are used to compute and analyze strategies in a discrete state-space.

To the best of our knowledge, apart from [20], we present the first algorithm that can solve timed games with large discrete state spaces. Although the algorithm applies to a subset of timed games and is not complete, we believe it is of utmost importance to make progress on the scalability of timed game solvers in order for these methods to be applied in convincing applications. Our paper makes an attempt in this direction.

We evaluate our algorithms in comparison with state-of-the-art tools and show that our approach is competitive with the existing tools, and can allow both model checking and synthesis to scale to larger models. The approach offers an alternative treatment of timed models, which might be applied in other settings.

A preliminary version of this paper appeared in [27].

We present the model checking algorithm in Section 2 which contains formal definitions, the description of the algorithm, and the experiments. Section 3 presents our contributions on the controller synthesis problem, and includes formal definitions, the description of the algorithm, and the experiments. In Section 4, we provide a broader discussion on related works, and present our conclusions and perspectives.

2 Compositional Model Checking

2.1 Preliminaries

Labeled Transition Systems and Finite Automata.

We denote *labeled transition systems* (LTS) as tuples $(Q, q^0, \Sigma \cup \{\epsilon\}, T)$ where Q is the set of states, $q^0 \in Q$ is the initial state, Σ is a finite alphabet, with ϵ a distinguished letter labeling silent transitions, $T \subseteq Q \times (\Sigma \cup \{\epsilon\}) \times Q$ is the transition relation. Because we will consider synchronous products of LTSs, we will use silent transitions to define internal transitions not exposed for synchronization. We will always make ϵ appear explicitly in considered alphabets. A *finite automaton* is an LTS given with a set of accepting states $F \subseteq Q$, and is written $(Q, q^0, \Sigma \cup \{\epsilon\}, T, F)$. A *run* of an automaton is a sequence $q_1 e_1 q_2 e_2 \dots q_n$, $e_i = (q_i, \sigma_i, q_{i+1}) \in T$ for some $\sigma_i \in \Sigma \cup \{\epsilon\}$ for each $1 \leq i \leq n-1$. The run is *initial* if $q_1 = q^0$. The *trace* of the run is the sequence $\sigma_1 \sigma_2 \dots \sigma_{n-1} \in \Sigma^*$. Here, ϵ is the neutral element; that is, the trace of a run does not contain ϵ . An *accepting run* starts at q^0 and ends in F . The language of a finite automaton $\mathcal{A}$ is the set of the traces of all accepting runs of $\mathcal{A}$, and is denoted by $\mathcal{L}(\mathcal{A})$. *Deterministic finite automata* (DFA) do not have silent transitions and have at most one edge for each label from each state.

We write $q \xrightarrow{\sigma} q'$ if there exists a run $q_1 \sigma_1 q_2 \sigma_2 \dots \sigma_{n-1} q_n$ from q to q' where there is a unique index $j \in \{1, 2, \dots, n-1\}$ such that $\sigma_j = \sigma$ and $\forall i \neq j, \sigma_i = \epsilon$.

The *parallel composition* of two LTSs $\mathcal{A}_i = (Q_i, q_i^0, \Sigma \cup \{\epsilon\}, T_i)$, $i \in \{1, 2\}$, defined on the same alphabet, is the LTS $\mathcal{A}_1 \parallel \mathcal{A}_2 = (Q, q^0, \Sigma \cup \{\epsilon\}, T, F)$ with $Q = Q_1 \times Q_2$, $q^0 = (q_1^0, q_2^0)$, and T contains $((q_1, q_2), \sigma, (q_1', q_2'))$ for all $(q_1, \sigma, q_1') \in T_1$, and $(q_2, \sigma, q_2') \in T_2$; and $((q_1, q_2), \epsilon, (q_1', q_2'))$ for all $(q_1, \epsilon, q_1') \in T_1$, and $q_2 \in Q_2$; and symmetrically, $((q_1, q_2), \epsilon, (q_1, q_2'))$ for all $(q_2, \epsilon, q_2') \in T_2$, and $q_1 \in Q_1$. The parallel composition of two automata $\mathcal{A}_i = (Q_i, q_i^0, \Sigma \cup \{\epsilon\}, T_i, F_i)$, $i \in \{1, 2\}$, is defined as the above LTS given with the accepting states $F = F_1 \times F_2$.

Finite Automata Learning.

We use finite automata learning algorithms such as L^* [24, 28] and TTT [25]. In the *online* learning model, the learning algorithm interacts with a teacher in order to learn a deterministic finite automaton recognizing a hidden regular language known to the teacher. The algorithm makes two types of queries. A *membership query* consists in asking whether a given word belongs to the language, to which the teacher answers by yes or no. An *equivalence query* consists in creating a hypothesis automaton H , and asking the teacher whether H recognizes the language. The teacher either answers yes, or no and provides a counterexample word which is in the symmetric difference of $\mathcal{L}(H)$ and of the target language. Learning algorithms typically make a large number of membership queries, and a smaller number of equivalence queries.

Timed Automata.

We fix a finite set of clocks $\mathcal{C}$. *Clock valuations* are the elements of $\mathbb{R}_{\geq 0}^{\mathcal{C}}$. For $R \subseteq \mathcal{C}$ and a valuation v , $v[R \leftarrow 0]$ is the valuation defined by $v[R \leftarrow 0](x) = v(x)$ for $x \in \mathcal{C} \setminus R$ and $v[R \leftarrow 0](x) = 0$ for $x \in R$. Given $d \in \mathbb{R}_{\geq 0}$ and a valuation v , $v + d$ is defined by

$(v + d)(x) = v(x) + d$ for all $x \in \mathcal{C}$. We extend these operations to sets of valuations in the standard way. We write $\vec{0}$ for the valuation that assigns 0 to every clock.

We consider a clock named 0 which has the constant value 0, and let $\mathcal{C}_0 = \mathcal{C} \cup \{0\}$. An *atomic guard* is a formula of the form $x \bowtie k$, or $x - y \bowtie k$ where $x, y \in \mathcal{C}_0$, $k, l \in \mathbb{N}$, and $\bowtie \in \{<, \leq, >, \geq\}$. A *guard* is a conjunction of atomic guards. A valuation v satisfies a guard g , denoted $v \models g$, if all atomic guards are satisfied when each $x \in \mathcal{C}$ is replaced by $v(x)$. Let $\Phi_{\mathcal{C}}$ denote the set of guards for $\mathcal{C}$.

Definition 1. A timed automaton $\mathcal{T}$ is a tuple $(L, \ell_0, \Sigma, \text{Inv}, \mathcal{C}, E, F)$, where L is a finite set of locations, $\ell_0 \in L$ is the initial location, Σ is the alphabet, $\text{Inv} : L \rightarrow \Phi_{\mathcal{C}}$ the invariants, $\mathcal{C}$ is a finite set of clocks, $E \subseteq L \times \Sigma \times \Phi_{\mathcal{C}} \times 2^{\mathcal{C}} \times L$ is a set of edges. An edge $e = (\ell, g, \sigma, R, \ell')$ is also written as $\ell \xrightarrow{g, \sigma, R} \ell'$. $F \subseteq L$ is the set of accepting locations.

We restrict to invariants that only contain upper bounds on clocks. Notice that we consider timed automata with alphabet without silent transitions. This is without loss of generality for our purpose; the reason behind this choice will be clear below when we define parallel composition with finite automata.

A *run* of $\mathcal{T}$ is a sequence $r = q_1 e_1 q_2 e_2 \dots q_n$ where $q_i \in L \times \mathbb{R}_{\geq 0}^{\mathcal{C}}$, and writing $q_i = (\ell, v)$ for each $1 \leq i \leq n$, we have $v \in \text{Inv}(\ell)$; if $i < n$, then either $e_i \in \mathbb{R}_{> 0}^{\mathcal{C}}$ and $v + e_i \in \text{Inv}(\ell)$, in which case $q_{i+1} = (\ell, v + e_i)$, or $e_i = (\ell, g, \sigma, R, \ell') \in E$, in which case $v \models g$ and $q_{i+1} = (\ell', v[R \leftarrow 0])$. The run is *initial* if $q_1 = (\ell_0, \vec{0})$. The run is *accepting* if the last location is in F . The *trace* of the run r is the word $\sigma_0 \sigma_1 \dots \sigma_n \in \Sigma^*$ where σ_i is the label of the i -th edge in the run. The *untimed language* of the timed automaton $\mathcal{T}$ is the set of the initial traces of the accepting runs of $\mathcal{T}$, and is denoted by $\mathcal{L}(\mathcal{T})$.

We define the parallel composition of timed automata as follows. Consider two timed automata $\mathcal{T}_i = (L_i, \ell_0^i, \Sigma_i, \text{Inv}_i, \mathcal{C}_i, E_i, F_i)$ assuming $\mathcal{C}_i$ are disjoint (otherwise we rename them). The parallel composition is the timed automaton $\mathcal{T}_1 \parallel \mathcal{T}_2 = (L_1 \times L_2, (\ell_0^1, \ell_0^2), \Sigma_1 \cup \Sigma_2, \text{Inv}, \mathcal{C}_1 \cup \mathcal{C}_2, E, F_1 \times F_2)$ where E contains the edge $((\ell_1, \ell_2), g_1 \wedge g_2, \sigma, R_1 \cup R_2, (\ell'_1, \ell'_2))$ whenever $\sigma \in \Sigma_1 \cap \Sigma_2$ and each $\mathcal{T}_i$ contains an edge $(\ell_i, g_i, \sigma, R_i, \ell'_i)$; furthermore, for each $\sigma \in \Sigma_1 \setminus \Sigma_2$ and edge $(\ell_1, g_1, \sigma, R_1, \ell'_1)$ in $\mathcal{T}_1$, and all ℓ_2 , $((\ell_1, \ell_2), g_1, \sigma, R_1, (\ell'_1, \ell_2))$ is in E ; and symmetrically for $\sigma \in \Sigma_2 \setminus \Sigma_1$. Last, $\text{Inv}((\ell_1, \ell_2)) = \text{Inv}_1(\ell_1) \wedge \text{Inv}_2(\ell_2)$. Parallel compositions of timed automata are called also called *networks of timed automata*.

We give a separate definition of the parallel composition of a finite automaton $\mathcal{A} = (Q, q^0, \Sigma \cup \{\epsilon\}, T, F)$ and a timed automaton $\mathcal{T} = (L, \ell_0, \Sigma, \text{Inv}, \mathcal{C}, E, F_{\mathcal{T}})$ which is a new timed automaton denoted by $\mathcal{A} \parallel \mathcal{T}$. Notice that the finite automaton is allowed to have ϵ -transitions. Intuitively, an edge labeled by σ is obtained by executing, in parallel, a σ -labeled edge of $\mathcal{T}$ and a sequence of edges in $\mathcal{A}$ performing $\xRightarrow{\sigma}$; while the guard and the reset of the overall edge are those of the edge of $\mathcal{T}$. Formally, let $\mathcal{A} \parallel \mathcal{T} = (L', \ell'_0, \Sigma, \text{Inv}', \mathcal{C}, E', F')$ with $L' = Q \times L$, $\text{Inv}' : (q, \ell) \mapsto \text{Inv}(\ell)$, $\ell'_0 = (q_0, \ell_0)$, and E' contains all edges of the form $((q, \ell), g, \sigma, R, (q', \ell'))$ such that $(\ell, g, \sigma, R, \ell') \in E$, and $q \xRightarrow{\sigma} q'$ in $\mathcal{A}$. We let $F' = F \times F_{\mathcal{T}}$.

Note that if we allowed $\mathcal{T}$ to have ϵ in the alphabet, a natural definition would be to define the parallel composition also allowing $\mathcal{T}$ to perform an arbitrary number of ϵ -transitions in one step. However, it would not be clear how to define the parallel composition as a timed automaton in this case since one cannot easily bound the number of ϵ -transitions to be allowed at each step; since clock values are not necessarily

repeated along a ϵ -transition cycle. If one does not wish to synchronize some of the edges of $\mathcal{T}$ with those of $\mathcal{A}$, one can label these with a fresh label τ in $\mathcal{T}$, and add self-loops with label τ to all states in $\mathcal{A}$.

When it comes to untimed languages, the parallel composition of a timed automaton and that of a finite automaton can be seen as an intersection, as shown by the next lemma.

Lemma 1. *Consider a finite automaton $\mathcal{A} = (Q, q^0, \Sigma \cup \{\epsilon\}, T, F)$ and a timed automaton $\mathcal{T} = (L, \ell_0, \Sigma, \text{Inv}, \mathcal{C}, E, F_{\mathcal{T}})$. We have $\mathcal{L}(\mathcal{A} \parallel \mathcal{T}) = \mathcal{L}(\mathcal{A}) \cap \mathcal{L}(\mathcal{T})$.*

Proof. Consider a trace $\sigma_1 \dots \sigma_n \in \mathcal{L}(\mathcal{A} \parallel \mathcal{T})$, and an accepting run

$$((q_1, \ell_1), v_1) e_1 ((q_2, \ell_2), v_2) e_2 \dots e_{m-1} ((q_m, \ell_m), v_m)$$

over this trace. First, by definition of the parallel composition, if we filter out delay transitions ($e_i \in \mathbb{R}_{\geq 0}$), then the remaining steps define a sequence $q_{i_1} \xrightarrow{\sigma_1} \dots \xrightarrow{\sigma_n} q_{i_{n+1}}$ with $q_{i_1} = q_1$ and $q_{i_{n+1}} = q_m$, hence $\mathcal{A}$ contains an accepting run over $\sigma_1 \dots \sigma_n$, that is, $\sigma_1 \dots \sigma_n \in \mathcal{L}(\mathcal{A})$.

Second, by the hypothesis that the alphabet of $\mathcal{T}$ does not contain ϵ , $\mathcal{T}$ must take a delay or an edge at each step in a run of the parallel composition. Thus, $(\ell_1, v_1) e_1 (\ell_2, v_2) e_2 \dots e_{m-1} (q_m, v_m)$ is a run of $\mathcal{T}$, which is obtained projecting the states of $\mathcal{A}$ away. It follows that $\sigma_1 \dots \sigma_n \in \mathcal{L}(\mathcal{T})$.

Conversely, assume that $\sigma_1 \dots \sigma_n \in \mathcal{L}(\mathcal{A})$ and $\sigma_1 \dots \sigma_n \in \mathcal{L}(\mathcal{T})$. There must exist an accepting run $q_1 \xrightarrow{\sigma_1} \dots \xrightarrow{\sigma_n} q_{n+1}$ in $\mathcal{A}$. Consider also a run in $\mathcal{T}$ over this trace:

$$(\ell_1, v_1) \delta_1 (\ell_1, v_1 + \delta_1) e_1 (\ell_2, v_2) \delta_2 (\ell_2, v_2 + \delta_2) e_2 \dots (\ell_{n+1}, v_{n+1})$$

written by alternating delay transitions $\delta_1, \delta_2, \dots \in \mathbb{R}_{\geq 0}$, and edges e_i labeled by σ_i 's. It follows that a run of $\mathcal{A} \parallel \mathcal{T}$ can be constructed by combining the two runs above:

$$((q_1, \ell_1), v_1) \delta_1 ((q_1, \ell_1), v_1 + \delta_1) e'_1 ((q_2, \ell_2), v_2) \delta_2 (\ell_2, v_2 + \delta_2) e'_2 \dots ((q_{n+1}, \ell_{n+1}), v_{n+1}),$$

establishing $\sigma_1 \dots \sigma_n \in \mathcal{L}(\mathcal{A} \parallel \mathcal{T})$. $\square$

Note that while analogous results are well known for the parallel composition of two finite automata, a similar result does not hold for networks of timed automata when considering untimed languages. Section 3.5.2 discusses the case where the two components are timed automata in which case we do not have equality but only inclusion in one direction.

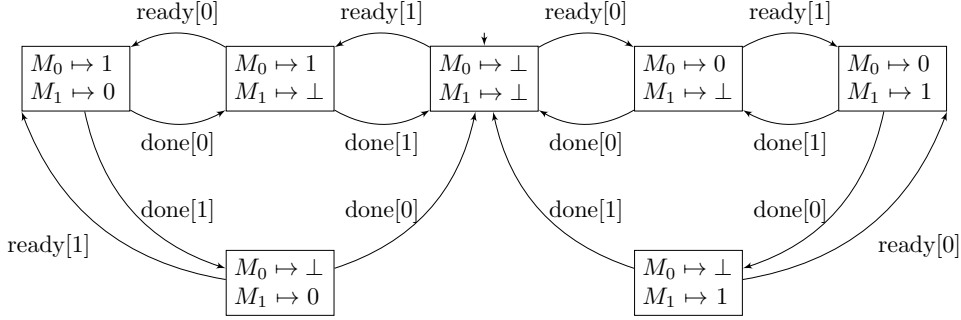
Target Timed Automata Instances.

Our main motivation is to consider real-time systems that are modeled naturally as $\mathcal{A} \parallel \mathcal{T}$. Typically, $\mathcal{A}$ has a large (discrete) state space, and $\mathcal{T}$ is a relatively small timed automaton, but with potentially complex time constraints involving several clocks.

An example is given in Figure 1 which shows how a simple scheduling setting can be modeled in this way. Here, the finite automaton is simple and only stores the mapping from machines to the tasks they are executing. Typically, if the machines or

the processes executing tasks have internal states, these could be modeled in $\mathcal{A}$ as well without altering the timed automaton.

Finite automaton $\mathcal{A}$:



Timed automaton $\mathcal{T}$:

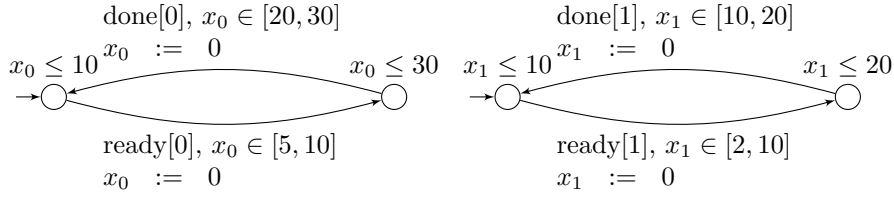


Fig. 1: Timed automaton $\mathcal{A} \parallel \mathcal{T}$ modeling a simple scheduling policy. The finite automaton $\mathcal{A}$ is given above and models a scheduler which schedules tasks (0 and 1) immediately when they become ready ($\text{ready}[0]$ and $\text{ready}[1]$) on machines M_0 and M_1 , using M_0 first if it is available. The timed automaton $\mathcal{T}$ is below, here, as a network of the timed automata, and models interarrival and computation times for each task.

2.2 Decomposing Timed Automata into $\mathcal{A} \parallel \mathcal{T}$

In this section, we prove that timed automata can be assumed to be in the form $\mathcal{A} \parallel \mathcal{T}$ without loss of generality. In other terms, all timed automata can be written in this form. Moreover, if the timed automaton $\mathcal{T}$ under study is not given as a composition, we show how to compute a decomposition $\mathcal{A} \parallel \mathcal{T}'$ with the same untimed language (up to projection). The construction we give here actually extends the alphabet, but as we show in Lemma 2, there is a projection of the untimed language of $\mathcal{A} \parallel \mathcal{T}'$ to the alphabet of $\mathcal{T}$ which preserves the untimed language of $\mathcal{T}$.

We will describe here an automatic translation from a “flat” timed automaton into the form $\mathcal{A} \parallel \mathcal{T}$. This not only establishes that this form can be assumed without loss of generality in this case, but also that it can be computed automatically (in linear time).

Before proceeding to the details of the translation, we would like to insist that we mostly see this translation as a theoretical result (the method can be applied to all timed automata without loss of generality) rather than being the first step of the algorithm described in this paper. In fact, we would like to advocate that in many

situations, the form $\mathcal{A} \parallel \mathcal{T}$ is the right way of modeling a real-time system. This clearly separates the modeling into two parts: the complex internal state space in $\mathcal{A}$ and real-time constraints with a simpler structure in $\mathcal{T}$. In Section 2.4, we designed most experiments ourselves while this structure naturally arose: e.g. in real-time distributed systems, $\mathcal{A}$ models the internal variables and control structure of the processes, while $\mathcal{T}$ models the communication medium. Mixing both would make the model awkward: for example, we would be using time intervals modeling communication latencies in the same transitions as those actually defining the process' behavior when receiving a message.

Consider a timed automaton $\mathcal{T}$. For each edge $(\ell, g, \sigma, R, \ell')$ in $\mathcal{T}$, we create a fresh label $\alpha_{(g \wedge \text{Inv}(\ell), \sigma, R)}$, and define the finite alphabet

$$G_{\mathcal{T}} = \{\alpha_{(g \wedge \text{Inv}(\ell), \sigma, R)} \mid (\ell, g, \sigma, R, \ell') \text{ is an edge of } \mathcal{T}\}.$$

Note that two edges sharing the same guard, invariant, label σ , and reset set yield the same label.

Let $\mathcal{A}$ be a finite automaton over alphabet $G_{\mathcal{T}}$ whose states are the locations of $\mathcal{T}$ and the edges are derived from those of $\mathcal{T}$ but labeled with corresponding labels from $G_{\mathcal{T}}$; more precisely, the edges are

$$\{(\ell, \alpha_{(g \wedge \text{Inv}(\ell), \sigma, R)}, \ell') \mid (\ell, g, \sigma, R, \ell') \text{ is an edge of } \mathcal{T}\}.$$

Now, define the timed automaton $\mathcal{T}'$ with a single state with the same clocks as $\mathcal{T}$, with one self-loop for each pair $\alpha_{(\phi, \sigma, r)} \in G_{\mathcal{T}}$: such an edge is labeled by $\alpha_{(\phi, \sigma, r)}$, has guard ϕ , and reset set R . Notice that g here stands for the conjunction of the guard of some edge and the invariant of the corresponding source location.

Define the following map $\text{proj} : G_{\mathcal{T}} \rightarrow \Sigma$ with $f(\alpha_{(\phi \wedge \text{Inv}(\ell), \sigma, R)}) = \sigma$.

Lemma 2. *Consider any timed automaton $\mathcal{T}$. Let $\mathcal{A} \parallel \mathcal{T}'$ denote the decomposition described above. Then $\mathcal{L}(\mathcal{T}) = \text{proj}(\mathcal{L}(\mathcal{A} \parallel \mathcal{T}'))$.*

Proof. Consider the following accepting run in $\mathcal{T}$:

$$(\ell_1, v_1) \delta_1 (\ell_1, v_1 + \delta_1) e_1 (\ell_2, v_2) \delta_2 (\ell_2, v_2 + \delta_2) e_2 \dots (\ell_{n+1}, v_{n+1})$$

where $\delta_i \geq 0$ are delay transitions and e_i 's are edges. Let us write $e_i = (\ell_i, g_i, \sigma_i, R_i, \ell_{i+1})$, and $\alpha_i = \alpha_{(g_i \wedge \text{Inv}(\ell_i), \sigma_i, R_i)}$. Let e'_i denote the edge of $\mathcal{A}$ obtained from e_i in the above construction. Then,

$$\ell_1 e'_1 \ell_2 e'_2 \dots \ell_{n+1}$$

is an accepting run in $\mathcal{A}$ by construction. Note that the trace of this run is $\alpha_1 \alpha_2 \dots \alpha_n$.

Let e_{α_i} denote the unique edge of $\mathcal{T}'$ labeled by α_i . We argue that one can take in $\mathcal{T}'$ the following sequence of delays and edge labels:

$$\delta_1 e_{\alpha_1} \delta_2 e_{\alpha_2} \dots e_{\alpha_n}.$$

In fact, each state $(\ell_i, v_i + \delta_i)$ satisfies the guard of e_i and the invariant of ℓ_i , which means that the guard of e_{α_i} (which is the conjunction of two) is satisfied. Furthermore, e_{α_i} has the same resets as e_i , so the same sequence of clock valuations is generated in both runs. This proves that $\alpha_1 \dots \alpha_n \in \mathcal{L}(\mathcal{A} \parallel \mathcal{T}')$, and therefore $\sigma_1 \dots \sigma_n \in \text{proj}(\mathcal{L}(\mathcal{A} \parallel \mathcal{T}'))$.

Conversely, consider a trace $\sigma_1 \dots \sigma_n \in \text{proj}(\mathcal{L}(\mathcal{A} \parallel \mathcal{T}'))$, and an accepting run of $\mathcal{A} \parallel \mathcal{T}'$:

$$((q_1, \ell_1), v_1) \delta_1 ((q_1, \ell_1), v_1 + \delta_1) e_{\alpha_1} ((q_2, \ell_2), v_2) \delta_2 ((q_2, \ell_2), v_2 + \delta_2) e_{\alpha_2} \dots ((q_{n+1}, \ell_{n+1}), v_{n+1}),$$

where delay transitions $\delta_i \geq 0$ and edge labels $\alpha_i = \alpha_{g_i \wedge \text{Inv}(\ell_i), \sigma_i, R_i}$ alternate such that $\text{proj}(\alpha_1 \dots \alpha_n) = \sigma_1 \dots \sigma_n$. By construction, for each i , there exists an edge $e_i = (\ell_i, g_i, \sigma_i, R_i, \ell_{i+1})$ in $\mathcal{T}$. It follows that the following run can be built in $\mathcal{T}$:

$$(\ell_1, v_1) \delta_1 (\ell_1, v_1 + \delta_1) e_1 (\ell_2, v_2) \delta_2 (\ell_2, v_2 + \delta_2) e_2 \dots (\ell_{n+1}, v_{n+1}).$$

In fact, because each $v_i + \delta_i$ satisfies the guard of e'_i , it satisfies the guard g_i of e_i and $\text{Inv}(\ell_i)$. Moreover, since invariants only contain upper bounds on clocks, v_i also satisfies $\text{Inv}(\ell_i)$. Hence $\sigma_1 \dots \sigma_n \in \mathcal{L}(\mathcal{T})$. $\square$

An example of this transformation is given in Fig. 2

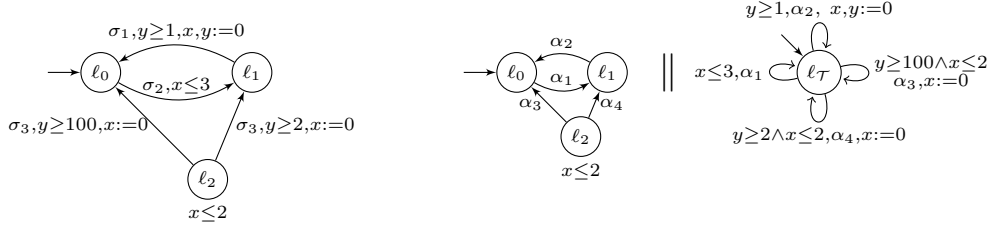


Fig. 2: The timed automaton $\mathcal{T}$ given on the left is transformed into the composition of $\mathcal{A} \parallel \mathcal{T}'$ given on the right. All guard, label, and reset triples are mapped to unique labels α_i here.

2.3 Learning-Based Compositional Model Checking Algorithm

We describe our algorithm for model checking the untimed language $\mathcal{L}(\mathcal{A} \parallel \mathcal{T})$.

Although it is known that the untimed language is regular [1], the size of the corresponding finite automaton can be exponential so a direct computation is not efficient. We will be looking for a finite automaton H which is an *overapproximation* of $\mathcal{T}$ i.e. $\mathcal{L}(\mathcal{T}) \subseteq \mathcal{L}(H)$. H stands for *hypothesis* made by the learning algorithm. We will in fact use the following lemma.

Lemma 3. *For all finite automata $\mathcal{A}$ and H , and timed automata $\mathcal{T}$ on common alphabet Σ , if $\mathcal{L}(\mathcal{T}) \subseteq \mathcal{L}(H)$, then $\mathcal{L}(\mathcal{A} \parallel \mathcal{T}) \subseteq \mathcal{L}(\mathcal{A} \parallel H)$.*

Proof. This follows from the observation that $\mathcal{L}(\mathcal{A} \parallel \mathcal{T}) = \mathcal{L}(\mathcal{A}) \cap \mathcal{L}(\mathcal{T})$, and $\mathcal{L}(\mathcal{A} \parallel H) = \mathcal{L}(\mathcal{A}) \cap \mathcal{L}(H)$. $\square$

In other terms, by replacing the timed automaton $\mathcal{T}$ by its overapproximation, we obtain an overapproximation of the compound system in terms of untimed language. So if a linear property can be established on $\mathcal{A} \parallel H$ for an appropriate H , then the property also holds on the original system.

Let us present the above property as a verification rule. Assuming that we want to establish $\mathcal{A} \parallel \mathcal{T} \subseteq \text{Spec}$ for some language Spec , we have

$$\frac{\mathcal{L}(\mathcal{T}) \subseteq \mathcal{L}(H) \quad \mathcal{L}(\mathcal{A} \parallel H) \subseteq \text{Spec}}{\mathcal{L}(\mathcal{A} \parallel \mathcal{T}) \subseteq \text{Spec.}} \text{ Asym} \quad (1)$$

Here, H serves as an *assumption* we make on $\mathcal{T}$ when verifying $\mathcal{A}$; so as in Lemma 3, we can use H instead of $\mathcal{T}$ during model checking. The rule (1) is well known as the assume-guarantee verification rule [29], and has been used in model checking finite-state systems as well as timed automata [30]. The assumption H can either be provided by the user, or automatically computed using automata learning as in [23]. Intuitively, the model checking algorithm we present in this section is an application of [23] to our specific case.

Figure 3 presents the overview of the algorithm. The membership queries of the learning algorithm are answered by the membership oracle; the equivalence query with conjecture H is answered by the inclusion oracle. When the conjecture H passes the inclusion check, we model-check $H \parallel \mathcal{A}$. When this is successful, we stop and declare that the original system $\mathcal{A} \parallel \mathcal{T}$ satisfies the specification. Otherwise, a counterexample $w \in \mathcal{L}(\mathcal{A} \parallel H) \setminus \text{Spec}$ was found, and we use a realizability check to see whether $w \in \mathcal{L}(\mathcal{T})$ (this is actually done by the membership oracle). If the answer is yes, then the counterexample is confirmed, and we stop. Otherwise, we inform the learning algorithm that w must be excluded, and continue the learning process.

Note that this algorithm can be used for any regular language specification Spec . We focus on safety properties in our experiments, presented next.

2.4 Experiments

We built a prototype implementation of our algorithm in Scala, using the TTT automata learning algorithm [25] from the learnlib library [31], and the associated automatalib for manipulating finite automata¹. We used the TChecker [4] model checker for implementing membership and inclusion oracles. For the latter, we complement H into H^c , and check the emptiness of the parallel composition of $\mathcal{T}$ with H^c . We use the NuSMV model checker for finite-state model checking.

The overall input consists in an SMV file describing $\mathcal{A}$, and of a TChecker timed automaton describing $\mathcal{T}$. We use define expressions in SMV to define the labels Σ , while TChecker allows us to tag each transition with a label.

¹<https://github.com/osankur/compRTMC/releases/tag/tacas23>

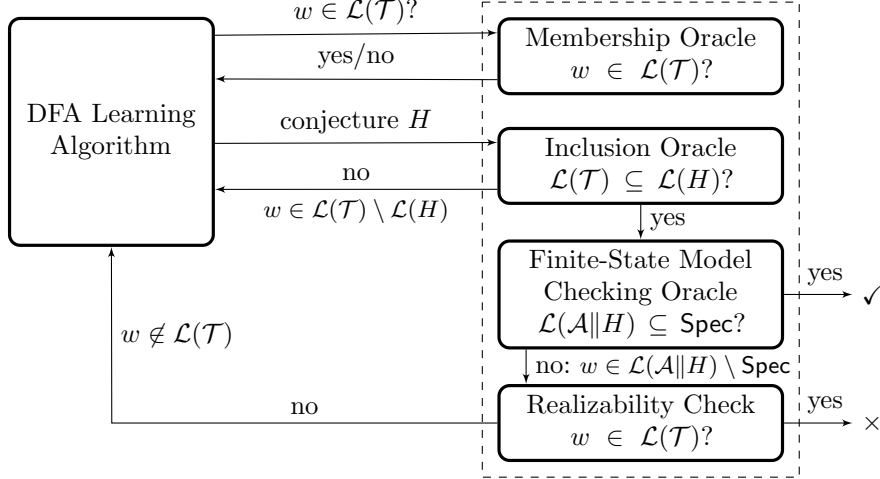


Fig. 3: The learning-based compositional model checking algorithm. The box on the left is a DFA learning algorithm, while the oracles answering the queries of the learning algorithm are shown on the right and correspond to the teacher.

We compare our algorithm on a set of benchmarks with the model checkers Uppaal [3] and nuXmv which has two algorithms for checking timed automata [13], both based on variants of ic3 [12, 32].

The former implements a zone-based enumerative algorithm, while the latter uses predicate abstraction through IC3IA.

Description of Benchmarks

The leader election protocol is a distributed protocol that can recover from crashes [33], extended here with periodic activation times and crash durations. The first four rows of Table 1 correspond to the case where one of the processes crashes when its internal state enters an error state. Internal states are modeled using Boolean circuits from the [synthesis competition](#) (SYNTCOMP) benchmarks. The stateless version is more abstract: there is no internal state model, and crashes can occur at any time. The letters A, B, C, D indicate different timed automaton models. Uppaal was more efficient at solving the stateless version but failed in the full version due to the large discrete state space. The compositional algorithm was effective in verifying all instances but the *D* case which required a large finite automaton to be learned. One can notice an overhead of the compositional algorithm in the stateless version due to the computation of the finite automaton H . This was particularly an issue in the stateless *D* case where Uppaal could find a counterexample trace faster. The default algorithm of nuXmv was not able to solve these instances, while the second algorithm could solve two instances but slower than our algorithm.

The flooding time synchronization protocol (FTSP) is a leader election algorithm for multi-hop wireless sensor networks used for clock synchronization [34], and has been the subject of formal verification before [35, 36]. We consider the abstract model used in

Table 1: Model checking benchmarks. The column #C is the number of clocks; #H is the number of hypothesis automata made by the DFA learning algorithm (in other terms, the number of conjectures); #M is the number of membership queries; and $|H|$ is the size of the final finite automaton learned. The safety specification holds on all models but those marked with *. In each cell, — means out of memory (8GB), and - means time out (30 minutes). The column nuXmv refers to the default algorithm [32] in the tool for checking timed automata, and nuXmv -a is the second algorithm based on predicate abstraction [12].

		Compositional				Uppaal	nuXmv	nuXmv -a
	#C	#H / #M	H	Time	Time	Time	Time	
Lead. Elec. A	3	13 / 232	15	157s	—	—	18m	
Lead. Elec. B	3	26 / 661	29	198s	—	—	-	
Lead. Elec. C	3	33 / 997	53	149s	—	-	6m46s	
Lead. Elec. D	3			-	—	-	-	
Lead. Elec. (simple) A	3	13 / 232	15	15s	6s	—	13m20s	
Lead. Elec. (simple) B	3	28 / 776	33	44s	8s	—	25m13s	
Lead. Elec. (simple) C	3	33 / 997	53	17s	6s	-	10m41s	
Lead. Elec. (simple) D*	3	134 / 6965	240	10m7s	6s	-	-	
FTSP-abstract-2	2	3 / 54	8	2s	2s	-	2m13s	
FTSP-abstract-3	3	17 / 340	23	47s	7m8s	-	-	
FTSP-abstract-4	4			-	-	-	-	
STS-2	5			7s	19s	-	-	
STS-3	6			-	-	-	-	
Rt-broadcast A	4	49 / 1324	63	59s	-	87s	9m3s	
Rt-broadcast B	4	41 / 1100	63	101s	-	90s	9m23s	
Rt-broadcast C	4	21 / 590	39	31s	-	86s	9m58s	
Rt-broadcast D	4	27 / 901	52	49s	-	80s	9m49s	
Priority Sched. 2 A	3	35 / 9859	49	34s	1s	7s	11s	
Priority Sched. 2 B	3	29 / 1162	42	16s	—	2s	6s	
Priority Sched. 3 C	4			-	—	6s	12s	
Priority Sched. 3 D	4			-	—	8s	39s	
Priority Sched. 3 E*	4			-	—	11s	16s	

[37] for parameterized verification allowing one to verify the model for a large number of topologies. Our algorithm was faster for the model with 3 processes, although none of the tools scaled to 4 processes.

The STS benchmarks are programming logic controller models from [38] and are part of the [TChecker benchmark database](#). These are known to be difficult to model check. The compositional algorithm was faster in STS-2, but none of the tools could solve STS-3.

The real-time broadcast protocol (rt-broadcast) implements a distributed algorithm made of $n = 3$ processes that wake up within a period interval, and stay active within a given time interval. If at least $m = 2$ of them are awake at the same time, they perform one step of a computation together and go back to sleep. The specification is whether a particular common configuration is reachable within a time bound. Uppaal could not solve these benchmarks. nuXmv could solve these although the compositional algorithm was often faster.

In the priority-based scheduling examples, a priority-based scheduling algorithm schedules tasks on a single machine. The interarrival times depend on the internal state of the processes which evolve over time. The internal states are modeled again by circuits from the SYNTCOMP benchmarks. The first two models have two tasks, and the last one has three. nuXmv performed very well on these benchmarks while the compositional algorithm failed on instances with three processes.

We designed ourselves all experiments but the STS. In all these, the form $\mathcal{A} \parallel \mathcal{T}$ arose naturally. In fact, timed automaton component is used to model activation times and communication latencies in the network, while $\mathcal{A}$ models the internal structure of each process. The STS models were the only exception: these were given as “flat” timed automata, and we used Lemma 2 to obtain the form $\mathcal{A} \parallel \mathcal{T}$.

Results

Overall, the experiments show that our algorithm is competitive with the state of the art tools; while it does not improve the performance uniformly on all considered benchmarks, it does allow us to solve instances that are not solvable by other tools, and sometimes to improve performance both compared to a zone-based approach (Uppaal) and SMT-based algorithms (nuXmv).

Let us comment the ratio between timed automata model checking and finite-state model checking times for the compositional algorithm. Because the models have large discrete state spaces and small timed automata components, the finite-state model checking step took significant time in most benchmarks. To give an example, the ratio of execution times spent solving membership queries (with TChecker) versus inclusion queries (NuSMV) was, respectively, 1%, 2%, 4% in the leader election benchmarks (first three rows), and 10%, 11%, 37% in the simple version. Notice that this difference is explained by the fact that the discrete part is much smaller in the simple benchmarks, so the ratios are naturally bigger compared to the full models. The ratio was 5% for the STS-2 benchmark. The priority scheduling 2 A and B cases had ratios 27% and 47% respectively, which were the highest. For the rt broadcast benchmarks, the ratio ranged from 17% to 29%. This ratio is difficult to predict: it depends on the efficiency of BDD-based model checking which we rely on, which is itself difficult to predict. The number of membership queries (that is, timed automata queries) varies here from several hundreds to several thousands and depends on the size of a minimal automaton that represents the untimed language of $\mathcal{T}$; this is also hard to predict.

While both types of queries had significant weight in the total execution time in some cases, there were few finite-state model checking queries and these were slow. In fact, Table 1 shows that a relatively small number of such queries ($\#H$) were done within the total execution time. In contrast, while a large number of timed automata model checking queries were run, these were typically quite fast (often around 10ms per query, and never above 100ms).

3 Compositional Controller Synthesis

3.1 Preliminaries

Games.

A *finite safety game* is a pair $(\mathcal{G}, \text{Bad})$ where $\mathcal{G}$ is an LTS $(Q_E \dot{\cup} Q_C, q_0, \Sigma, T)$ with the set of states given as a partition $Q_E \dot{\cup} Q_C$, namely, *Environment states* (Q_E), and *Controller states* (Q_C), and $\text{Bad} \subseteq Q_E \dot{\cup} Q_C$ is an *objective*. The game is played between two players, namely, *Controller* and *Environment*. At each state $q \in Q_C$, Controller determines the successor by choosing an edge from q , and Environment determines the successor from states $q \in Q_E$. A *strategy* for Controller (resp. Environment) maps finite runs of $(Q_E \dot{\cup} Q_C, q_0, \Sigma, T)$ ending in Q_C (resp. Q_E) to an edge leaving the last state. A pair of strategies, one for each player, induces a unique infinite run from the initial state. A run is *winning* for Controller if it does not visit Bad ; it is winning for Environment otherwise. A *winning strategy* for Controller is such that for all Environment strategies, the run induced by the two strategies is winning for Controller. Symmetrically, Environment has a winning strategy if for all Controller strategies, the induced run is winning. A strategy is *positional* (a.k.a. memoryless) if it only depends on the last state of the given run.

The parallel composition of $(\mathcal{G}, \text{Bad})$ and a deterministic finite automaton $\mathcal{F} = (Q', q'_0, \Sigma, T', F)$ on alphabet Σ is a new game whose LTS is $\mathcal{G} \parallel \mathcal{F}$ in which the Controller states are $Q_C \times Q'$, the Environment states are $Q_E \times Q'$, and the objective is $\text{Bad} \times F$ (Notice that Controller thus has a safety objective).

Finite games were extended to the real-time setting as *timed games* [16, 17]. A *timed game* is a timed automaton $\mathcal{T} = (L_E \dot{\cup} L_C, \ell_0, \Sigma, \text{Inv}, \mathcal{C}, E, \text{Bad})$ with the exception that its edges are labeled by $\Sigma \cup \{\epsilon\}$ (and not just by Σ as in the previous section), and the locations are partitioned as $L_E \dot{\cup} L_C$ into *Environment locations* and *Controller locations*. The semantics is defined by letting Environment choose the delay and the edge to be taken at locations L_E , while Controller choose these from L_C . Formally, a *strategy* for Environment (resp. Controller) is a function which associates a run that ends in L_E (resp. L_C) to a pair of delay and an edge enabled from the state reached after the delay. A run is winning for Controller if it does not visit Bad . A Controller (resp. Environment) strategy is *winning* for objective Bad if for all Environment (resp. Controller) strategies, the induced run from the initial state is winning (resp. not winning) for Controller. A run r is *compatible* with a strategy $\mathfrak{S}$ for Controller (resp. Environment) if there exists an Environment (resp. Controller) strategy $\mathfrak{S}'$ such that r is induced by $\mathfrak{S}, \mathfrak{S}'$.

The parallel composition of a finite safety game $(\mathcal{G}, \text{Bad})$ and a timed automaton $\mathcal{T} = (L, \ell_0, \Sigma, \text{Inv}, \mathcal{C}, E, F)$ on common alphabet Σ is the timed game $\mathcal{G} \parallel \mathcal{T}$ where Controller locations are $Q_C \times L$; Environment locations are $Q_E \times L$; and the objective is $\text{Bad} \times F$.

Observe that players select edges in the games we consider, and the edge labels are only used for synchronizing $\mathcal{G}$ and $\mathcal{T}$. Thus, the determinism of $\mathcal{G} \parallel \mathcal{T}$ is not an issue.

Positional strategies exist both for reachability and safety objectives in finite and timed games. Both finite and timed games are known to be *determined* for reachability and safety objectives. For instance, if Controller does not have a winning strategy for

the safety objective, then Environment has a strategy ensuring the reachability of **Bad** [16, 17].

Target Timed Game Instances.

We consider controller synthesis problems described as timed games in the form of $(\mathcal{G} \parallel \mathcal{T}, \text{Bad} \times F)$ where $(\mathcal{G}, \text{Bad})$ is a finite safety game, and $\mathcal{T}$ is a timed automaton with accepting states F . In addition, we assume that $\mathcal{G} \parallel \mathcal{T}$ is *Controller-silent*, defined as follows.

Definition 2. *The timed game $(\mathcal{G} \parallel \mathcal{T}, \text{Bad} \times F)$ on alphabet Σ is Controller-silent if all Controller transitions are silent.*

Hence, we again separate the game $\mathcal{G}$ defined on a possibly large discrete state space while real-time constraints are separately given in $\mathcal{T}$.

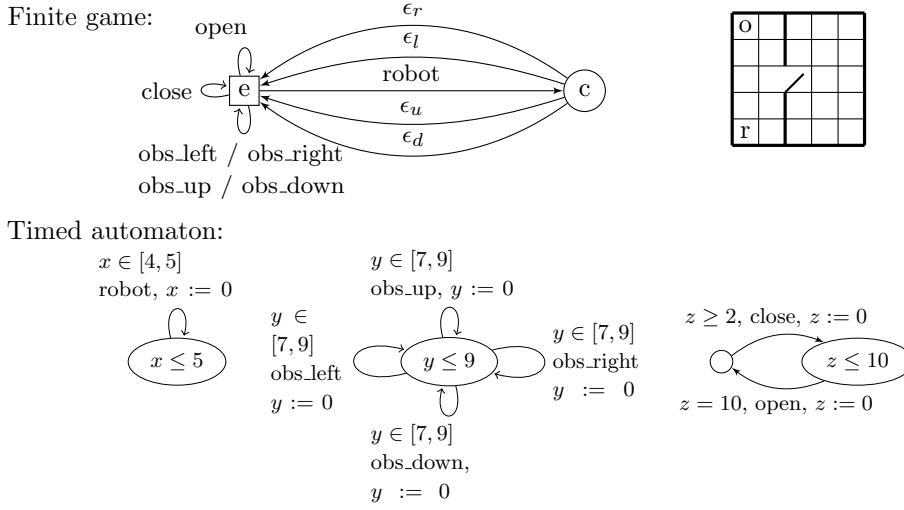


Fig. 4: The sketch of a timed game $\mathcal{G} \parallel \mathcal{T}$ modelling a planning problem. The finite game models a robot and an obstacle moving in a grid world as shown on top right. The cells r and o show, respectively, the initial positions of the robot and the obstacle. The robot cannot cross walls (shown in thick segments), and can only cross the door if it is open. Here four silent transitions were marked with $\epsilon_r, \epsilon_l, \epsilon_u, \epsilon_d$ for readability; in reality, these are all labeled by ϵ .

The intuition behind the semantics is the following: because the game is played in $\mathcal{G} \parallel \mathcal{T}$ and $\mathcal{G}$ is Controller-silent, the timed automaton model $\mathcal{T}$ is only used to disallow some of the Environment transitions according to real-time constraints, while Controller's actions are responses to Environment's actions and thus are unaffected by the constraints of $\mathcal{T}$. One can think of the timed automaton as some form of scheduler that schedules uncontrollable events in the system, so the order of these is determined by Environment. It is useful to think that Controller responds instantaneously to Environment (with 0 delay) although it can actually use positive delays. This asymmetric

view enables a one-sided abstraction framework presented in the next section, where Environment transitions are approximated by a DFA.

An example is given in Figure 4. The finite game drawn here only shows the structure of the game. It has, in addition, integer variables `rob_x`, `rob_y`, `obs_x`, `obs_y` encoding the positions of the robot and of the obstacle, and a Boolean variable `door` to encode the state of the door. The state `e` belongs to Environment, which can move the obstacle in any direction, close or open the door, or let the robot move by going to state `c`. The state `c` belongs to Controller. All its leaving transitions are silent, and correspond to moving the robot in four directions. These transitions have preconditions, not shown in the figure, that check whether the moves are possible, and have updates that modify the state variables. The timed automaton, given as a network of three timed automata, determine the timings of these events. One can notice, for example, that the robot is moving faster than the obstacle, and that whenever the door is closed, it remains so for 10 time units.

3.2 One-Sided Abstraction

Thanks to the assumption we make on considered timed games, we show that by replacing $\mathcal{T}$ by a DFA H that is an overapproximation, we obtain an abstract game in which Controller strategies can be transferred to the original game.

This is formalized in the next lemma that shows that it is sufficient to look for Controller strategies in the abstract game.

Lemma 4. *Consider a Controller-silent timed game $(\mathcal{G} \parallel \mathcal{T}, \text{Bad} \times F)$, and a complete DFA H with accepting states F_H , satisfying $\mathcal{L}(\mathcal{T}) \subseteq \mathcal{L}(H)$. If Controller wins $(\mathcal{G} \parallel H, \text{Bad} \times F_H)$, then it wins $(\mathcal{G} \parallel \mathcal{T}, \text{Bad} \times F)$.*

Proof. Consider a winning strategy $\mathfrak{S}$ for Controller in $(\mathcal{G} \parallel H, \text{Bad} \times F_H)$.

Let us define a set A of runs of $\mathcal{G} \parallel \mathcal{T}$ as well as a mapping f from A to the runs of $\mathcal{G} \parallel H$. We will at the same time define a strategy $\mathfrak{S}'$ on $\mathcal{G} \parallel \mathcal{T}$ whose compatible runs will be A . We will do so by imitating $\mathfrak{S}$ using the mapping f . We define f on the runs inductively for increasing lengths.

- Initially, we let $f((q_0^{\mathcal{G}}, q_0^{\mathcal{T}})) = (q_0^{\mathcal{G}}, q_0^H)$ where $q_0^{\mathcal{T}}$ is the initial state of $\mathcal{T}$, and $q_0^{\mathcal{G}}$ that of $\mathcal{G}$, and respectively for q_0^H and H .
- For $n \geq 0$, consider a run $\rho = (q_0^{\mathcal{G}}, q_0^{\mathcal{T}}) \dots (q_n^{\mathcal{G}}, q_n^{\mathcal{T}})$ of A and assume that $f(\rho) = (q_0^{\mathcal{G}}, q_0^H) \dots (q_n^{\mathcal{G}}, q_n^H)$ is already defined. We distinguish two cases.
 - If $(q_n^{\mathcal{G}}, q_n^{\mathcal{T}})$ belongs to Environment, then for all delays $\delta \geq 0$, and edges e enabled from $q_n^{\mathcal{T}}$ after the selected delay, the successor $(q_{n+1}^{\mathcal{G}}, q_{n+1}^{\mathcal{T}})$, we add $\rho \cdot (q_{n+1}^{\mathcal{G}}, q_{n+1}^{\mathcal{T}})$ to A . If σ denotes the label of the edge e , we further define $f(\rho \cdot (q_{n+1}^{\mathcal{G}}, q_{n+1}^{\mathcal{T}})) = (q_0^{\mathcal{G}}, q_0^H) \dots (q_n^{\mathcal{G}}, q_n^H)(q_{n+1}^{\mathcal{G}}, q_{n+1}^H)$ where q_{n+1}^H is the unique successor of q_n^H with label $\sigma \in \Sigma$, and $q_{n+1}^H = q_n^H$ if $\sigma = \epsilon$. Such a successor exists because H is complete, and it is unique because H is deterministic.
 - Otherwise, $(q_n^{\mathcal{G}}, q_n^{\mathcal{T}})$ belongs to Controller, and we consider the move prescribed by $\mathfrak{S}$ at $(q_0^{\mathcal{G}}, q_0^H) \dots (q_n^{\mathcal{G}}, q_n^H)$ in $\mathcal{G} \parallel H$. This consists in a silent edge e of $\mathcal{G}$ by assumption. Let $q_{n+1}^{\mathcal{G}}$ be the successor of $q_n^{\mathcal{G}}$ through e . We add $\rho \cdot (q_{n+1}^{\mathcal{G}}, q_n^{\mathcal{T}})$ to A , and define $f(\rho \cdot (q_{n+1}^{\mathcal{G}}, q_n^{\mathcal{T}})) = (q_0^{\mathcal{G}}, q_0^H) \dots (q_n^{\mathcal{G}}, q_n^H)(q_{n+1}^{\mathcal{G}}, q_n^H)$ (recall that

$\mathcal{T}$ does not change states on a silent transition, nor does H). In this case, we let $\mathfrak{S}'$ choose this move from ρ after a 0 delay (note that because the timed automaton location has not changed, the new state still satisfies the invariant). Notice that if $f(\rho) = \pi$, then these have the same trace and their first components are identical.

Consider any infinite run ρ of $\mathcal{G} \parallel \mathcal{T}$ compatible with $\mathfrak{S}'$. For any finite prefix ρ' of ρ , $f(\rho')$ is a run of $\mathcal{G} \parallel H$ compatible with $\mathfrak{S}$. By assumption, $f(\rho')$ does not end in $\text{Bad} \times F_H$. This means that either the first component is not in Bad , or the second component is not in F_H . In the first case, this is also the case for ρ' . In the second case, the trace of $f(\rho')$ is not in $\mathcal{L}(H)$. But then the trace of ρ' is not in $\mathcal{L}(\mathcal{T})$ by $\mathcal{L}(\mathcal{T}) \subseteq \mathcal{L}(H)$. Therefore, ρ is winning for Controller. $\square$

We do not have the symmetrical property for Environment due to assymetric assumption on $\mathcal{T}$ being Controller-silent. However, we show that if Environment wins the original game $\mathcal{G} \parallel \mathcal{T}$, then it wins the abstract game while generating the same traces: **Lemma 5.** *Consider a Controller-silent timed game $(\mathcal{G} \parallel \mathcal{T}, \text{Bad} \times F)$, and a complete DFA H with accepting states F_H , satisfying $\mathcal{L}(\mathcal{T}) \subseteq \mathcal{L}(H)$. If Environment wins $(\mathcal{G} \parallel \mathcal{T}, \text{Bad} \times F)$, then it wins $(\mathcal{G} \parallel H, \text{Bad} \times F_H)$, and has a strategy in $(\mathcal{G} \parallel H, \text{Bad} \times F_H)$ whose all compatible runs have traces in $\mathcal{L}(\mathcal{T})$.*

Proof. The first part of the statement follows from the determinacy of the games we consider. In fact, if Environment does not win $(\mathcal{G} \parallel H, \text{Bad} \times F_H)$, then Controller wins this game. By Lemma 4, Controller then wins $(\mathcal{G} \parallel \mathcal{T}, \text{Bad} \times F)$. So by the determinacy of timed games, Environment loses $(\mathcal{G} \parallel \mathcal{T}, \text{Bad} \times F)$. This establishes the first part by its contrapositive.

It remains to show that when Environment wins in $(\mathcal{G} \parallel \mathcal{T}, \text{Bad} \times F)$ with strategy $\mathfrak{S}$, then it has a strategy in $(\mathcal{G} \parallel H, \text{Bad} \times F_H)$ whose all compatible runs have their traces in $\mathcal{L}(\mathcal{T})$. This can be done by a construction that is similar to the first case. We define a set B of runs of $\mathcal{G} \parallel H$ and a mapping g from B to the runs of $\mathcal{G} \parallel \mathcal{T}$. We also define $\mathfrak{S}'$ as an Environment strategy in $\mathcal{G} \parallel H$ whose compatible runs are exactly B .

- First, B contains the initial state $(q_0^{\mathcal{G}}, q_0^H)$ and we define $g((q_0^{\mathcal{G}}, q_0^H)) = (q_0^{\mathcal{G}}, q_0^{\mathcal{T}})$.
- Consider a run $\rho = (q_0^{\mathcal{G}}, q_0^H) \dots (q_n^{\mathcal{G}}, q_n^H)$ with $n \geq 0$ in B on which g is already defined.
 - Assume that it ends in an Environment state. We consider the delay and edge (δ, e) prescribed by $\mathfrak{S}$ to $g(\rho)$. Let $(q_{n+1}^{\mathcal{G}}, q_{n+1}^{\mathcal{T}})$ be the successor of $g(\rho)$ after (δ, e) . Let q_{n+1}^H be the unique successor of q_n^H on the label of e , and $q_{n+1}^H = q_n^H$ if the label is ϵ . We add $\rho(q_{n+1}^{\mathcal{G}}, q_{n+1}^H)$ to B , and define $g(\rho(q_{n+1}^{\mathcal{G}}, q_{n+1}^H)) = g(\rho)(q_{n+1}^{\mathcal{G}}, q_{n+1}^{\mathcal{T}})$. We define $\mathfrak{S}'$ so that it assigns $(q_{n+1}^{\mathcal{G}}, q_{n+1}^H)$ to ρ .
 - If ρ ends in a Controller state, then consider any delay $\delta \geq 0$ and edge e whose guard is satisfied after a delay of δ from $g(\rho)$. Note that e has label ϵ by assumption. Consider the state $(q_{n+1}^{\mathcal{G}}, q_{n+1}^{\mathcal{T}})$ reached from $g(\rho)$ after delay δ and edge e . The choice of the same edge from ρ leads to some state $(q_{n+1}^{\mathcal{G}}, q_n^H)$; we add $\rho(q_{n+1}^{\mathcal{G}}, q_n^H)$ to B , and define $g(\rho(q_{n+1}^{\mathcal{G}}, q_n^H)) = g(\rho)(q_{n+1}^{\mathcal{G}}, q_n^{\mathcal{T}})$.

It follows that $\mathfrak{S}'$ is winning for Environment in $\mathcal{G} \parallel H$; in fact, g always maps to states from which $\mathfrak{S}$ is winning. As in the proof of Lemma 4, $\mathfrak{S}$ and $\mathfrak{S}'$ induce the same traces by the mapping g , so we have that all compatible runs under $\mathfrak{S}'$ are in $\mathcal{L}(\mathcal{T})$. $\square$

Note that in the above lemma, it is crucial that the game is Controller-silent. In fact, if Controller could take edges that synchronize with $\mathcal{T}$, then we may not be able to apply a strategy in $\mathcal{G} \parallel H$ to $\mathcal{G} \parallel \mathcal{T}$, since such a strategy may prescribe traces that are not accepted in $\mathcal{T}$. We illustrate this in the following example.

Example 1. Consider the timed game $(\mathcal{G} \parallel \mathcal{T}, \text{Bad} \times F)$ given in Fig. 5. Here, square states show Environment states, and circle ones show Controller states. The objective of Controller is to avoid the state bad, which means avoiding the edge label d. However Environment can ensure reaching bad, as follows. Initially all clocks are 0. We have that the delay picked by Controller at the first step is 0 if, and only if $z - x = 0$; and the delay picked by Environment in the second step is 0 if, and only if $x - y = 0$. The edge d is only enabled if Environment has made a 0 delay but not Controller, or vice versa. Thus, Environment can ensure that the edge d is enabled, and that the edge c is disabled in $\mathcal{T}$, by choosing a zero or a nonzero delay depending on the choice of Controller. The resulting runs starts with abd and visits bad.

Consider now a DFA overapproximation H of $\mathcal{T}$ which recognizes the language $(ab(c+d))^*$, for instance, obtained from $\mathcal{T}$ by removing all clocks. We do have $\mathcal{L}(\mathcal{T}) \subseteq \mathcal{L}(H)$. But Controller has a winning strategy in $\mathcal{G} \parallel H$ since it can ensure $(abc)^*$.

3.3 Learning-Based Compositional Controller Synthesis

We now present our compositional controller synthesis algorithm whose overview is given in Figure 6. The algorithm for controller synthesis is more involved than the model checking algorithm due to the alternating semantics for two players in games. It consists in two phases that alternate: the overapproximation phase, and the underapproximation phase. Each phase runs a DFA learning algorithm which is interrupted when we switch to the other phase, and continued when we switch back, until a decision is made. Together, both phases maintain two approximations, $\underline{H}$ and $\overline{H}$, such that $\mathcal{L}(\underline{H}) \subseteq \mathcal{L}(\mathcal{T}) \subseteq \mathcal{L}(\overline{H})$.

The objective of the overapproximation phase is to attempt to learn a DFA $\overline{H}$ satisfying $\mathcal{L}(\mathcal{T}) \subseteq \overline{H}$, and such that Controller wins in $\mathcal{G} \parallel \overline{H}$. The learning algorithm uses membership and inclusion oracles just like in Section 2.3. Once such a candidate

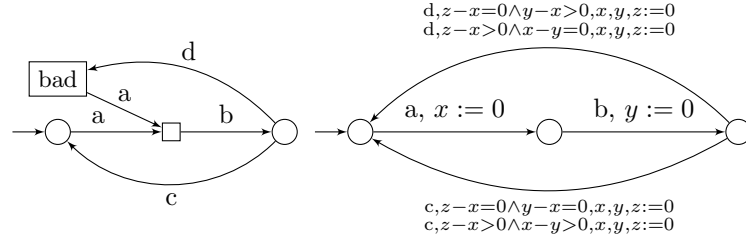


Fig. 5: A timed game $(\mathcal{G} \parallel \mathcal{T}, \text{Bad} \times F)$ with clocks x, y, z that is not Controller-silent, where $\mathcal{G}$ is given on the left, and $\mathcal{T}$ on the right, with $\text{Bad} = \{\text{bad}\}$ and F contains all states of $\mathcal{T}$. There are two transitions labeled by c in $\mathcal{T}$ shown on a single edge; here, each line corresponds to the label, guard, and the resets of a transition, and similarly for the two transitions labeled by d .

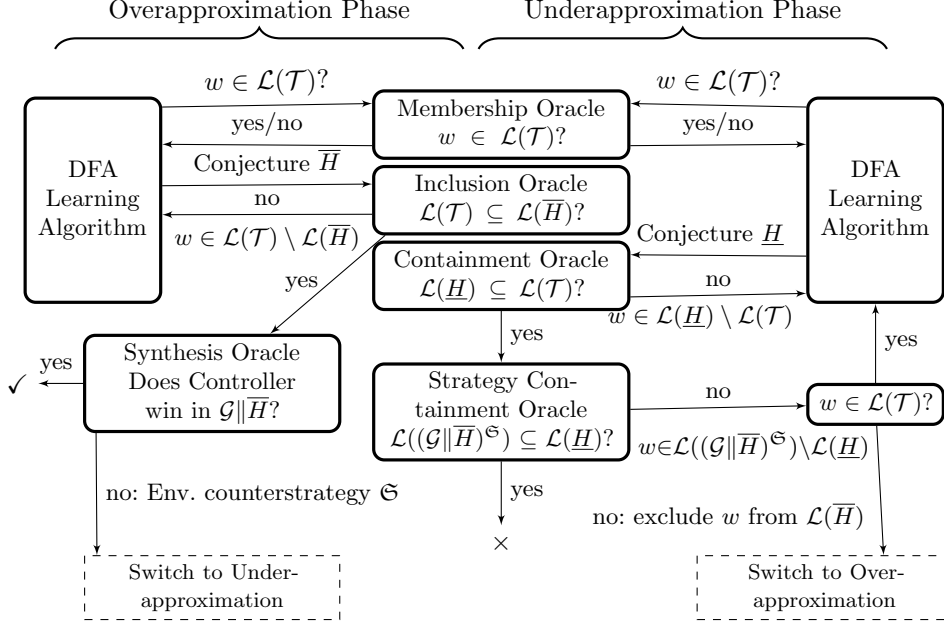


Fig. 6: The learning-based compositional controller synthesis algorithm for the input timed game $\mathcal{G} \parallel \mathcal{T}$, with $\mathcal{G}$ a Controller-silent finite game, and $\mathcal{T}$ a label-deterministic timed automaton. Two automata learning algorithms run in parallel to learn under- and over-approximations $\underline{H}$ and $\overline{H}$ such that $\underline{H} \subseteq \mathcal{L}(\mathcal{T}) \subseteq \overline{H}$.

DFA $\overline{H}$ is found, the synthesis oracle checks, using finite-state techniques, whether Controller has a winning strategy in $\mathcal{G} \parallel \overline{H}$. If this is the case, we stop and conclude that Controller wins in $\mathcal{G} \parallel \mathcal{T}$ by Lemma 4. Otherwise, Environment has a winning strategy $\mathfrak{S}$ in this game; and we switch to the underapproximation phase.

The goal of the underapproximation is to check whether the given Environment strategy $\mathfrak{S}$ can be proved to be spurious. Intuitively, we would like to check whether $\mathcal{L}((\mathcal{G} \parallel \overline{H})^{\mathfrak{S}}) \subseteq \mathcal{L}(\mathcal{T})$ and reject if this is the case. In fact, by Lemma 5, we know that a winning Environment strategy in $\mathcal{G} \parallel \mathcal{T}$ implies that there is such a strategy $\mathfrak{S}$. This is the source of incompleteness of our algorithm, since this condition is necessary but not sufficient for Environment to win; that is, the condition does not guarantee that Environment actually wins in $\mathcal{G} \parallel \mathcal{T}$.

While $\mathcal{L}((\mathcal{G} \parallel \overline{H})^{\mathfrak{S}}) \subseteq \mathcal{L}(\mathcal{T})$ can be checked with a timed automaton model checker (see *Checking Containment* below), this would mean exploring the large state space due to $\mathcal{G}$. Since we want to avoid using timed automata model checkers on such large instances, we rather learn an underapproximation $\underline{H}$ of $\mathcal{L}(\mathcal{T})$ using the membership and containment oracles, and use a finite-state model checker to check $\mathcal{L}((\mathcal{G} \parallel \overline{H})^{\mathfrak{S}}) \subseteq \mathcal{L}(\underline{H})$. Note that although the learning process does require inclusion checks of the form $\mathcal{L}(\underline{H}) \subseteq \mathcal{L}(\mathcal{T})$, this check is feasible with a timed automaton model checker since $\underline{H}$ is typically much smaller than $\mathcal{G}$. If the above check passes, then we reject the instance, that is, we declare the system not controllable. Otherwise, some trace w

appears in $\mathcal{L}((\mathcal{G} \parallel \overline{H})^\mathfrak{S})$ but not in $\mathcal{L}(\underline{H})$. If $w \in \mathcal{L}(\mathcal{T})$, then we require that w be included in $\underline{H}$, and continue the learning process. Otherwise, $\mathfrak{S}$ is not valid since it induces w which is not in $\mathcal{L}(\mathcal{T})$. So we interrupt the current phase and switch back to the overapproximation phase requiring w to be removed from $\overline{H}$.

Membership and inclusion oracles are implemented with a timed automata model checker. Here, the synthesis oracle can be any finite game solver; we just need the capability of computing the controlled system $(\mathcal{G} \parallel \overline{H})^\mathfrak{S}$. Such a system is finite-state, so the strategy containment oracle can be implemented using a finite-state model checker (since $\underline{H}$ is deterministic and can thus be complemented). It remains to explain how the containment oracle is implemented.

Checking Containment $\mathcal{L}(\underline{H}) \subseteq \mathcal{L}(\mathcal{T})$.

First, notice that, even with determinism assumptions on $\mathcal{T}$, the untimed language of the timed automaton complement of $\mathcal{T}$ is not the complement of $\mathcal{L}(\mathcal{T})$. To see this, consider a timed automaton with a single state which is both initial and accepting, a single clock x , and a self-loop with guard $x = 1$, labeled by σ . Then, both $\mathcal{L}(\mathcal{T})$ and $\mathcal{L}(\mathcal{T}^c)$ are the language σ^* where $\mathcal{T}^c$ denotes the timed automaton complement.

A timed automaton is *label-deterministic* if at each location ℓ , for each label $\sigma \in \Sigma$, there is at most one transition leaving ℓ labelled by σ ; in other terms, the finite automaton obtained by removing all clocks is deterministic.

Assuming the label-determinism of $\mathcal{T}$, this check can be done by a simple adaptation of a zone-based exploration algorithm, as follows. Let us assume that accepting states are reachable from all states of $\underline{H}$, which can be ensured by a preprocessing step. We start exploring the timed automaton $\underline{H} \parallel \mathcal{T}$ using a zone-based exploration algorithm [39]. Consider any search node $((q_{\underline{H}}, q_{\mathcal{T}}), Z)$ encountered during the exploration algorithm, reachable by the trace w , where $(q_{\underline{H}}, q_{\mathcal{T}})$ is a location of $\underline{H} \parallel \mathcal{T}$, and Z a zone. The exploration algorithm generates all available successors for $\sigma \in \Sigma$. We make the following additional check: If there is $\sigma' \in \Sigma$ such that $q_{\underline{H}}$ has a successor by σ' , but not $\mathcal{T}$ (either because there is no such edge, or because the guard of the unique edge labeled by σ' is not satisfied by Z), then we stop and return the trace $w\sigma' \in \mathcal{L}(\underline{H}) \setminus \mathcal{L}(\mathcal{T})$ as a counterexample to containment. If no such label can be found, the zone-based exploration terminates and confirms the containment.

It is easy to see that this algorithm determines if $\mathcal{L}(\underline{H}) \subseteq \mathcal{L}(\mathcal{T})$. In fact, if a counterexample $w\sigma'$ is returned, then w can indeed be read in both automata. Assume that the search reaches $((q_{\underline{H}}, q_{\mathcal{T}}), Z)$ after reading w , through a sequence of edges $e_1 e_2 \dots e_n$. Because $\mathcal{T}$ is label-deterministic, this is the only edge sequence that can be taken in $\mathcal{T}$ while reading w . Moreover, by the properties of zone-based exploration [39], Z is the strongest postcondition of this edge sequence, that is, it contains all possible clock valuations that can be reached by reading $e_1 e_2 \dots e_n$. Because no edge with label σ' is feasible from here, this means that $w\sigma'$ cannot be read in $\mathcal{T}$, while it can in $\underline{H}$. Because accepting states are reachable from all states of $\underline{H}$, $w\sigma'$ can be extended to an accepting word; therefore $\mathcal{L}(\underline{H}) \not\subseteq \mathcal{L}(\mathcal{T})$. Conversely, if $\mathcal{L}(\underline{H}) \subseteq \mathcal{L}(\mathcal{T})$, then let $w\sigma'$ denote a minimal counterexample where w can be read in $\underline{H}$ (but not necessarily reaching an accepting state), but not in $\mathcal{T}$. The algorithm will eventually

pick the sequence of edges with labels w , and reach some state $((q_H, q_T), Z)$, where σ' is available in $\underline{H}$ but not $\mathcal{T}$, so a counterexample will be returned.

As an alternative, one can use testing such as the Wp-method [40] to establish the containment, as it is customary in DFA learning. In this case, the answer is approximate in the sense that the conformance test can fail to detect that containment does not hold. However, this does not affect the soundness of the overall algorithm since it can only increase false negative for the controller synthesis problem. This is what we use in our experiments, presented next.

3.4 Experiments

Our tool accepts instances $\mathcal{G} \parallel \mathcal{T}$ where $\mathcal{G}$ is given as a Verilog module, and $\mathcal{T}$ as a TChecker timed automaton. Some of the inputs of the Verilog module are *uncontrollable* (chosen by Environment), some others are controllable (chosen by Controller). We use outputs of the Verilog module to define the synchronization labels Σ ; while TChecker models tag each transition with such a label.

Table 2: The results of the controller synthesis experiments. The columns #C, #H, #M respectively show the number of clocks in the model, the numbers of conjectures and membership queries made by the compositional algorithm; while $|\overline{H}|$, $|\underline{H}|$ show the sizes of the DFAs learned by the two phases.

	#C	Compositional Algorithm					TIGA	Answer
		#H	#M	$ \overline{H} $	$ \underline{H} $	Time	Time	
Scheduling genbuf A	3	50	2178	114		26s	mo	yes
Scheduling genbuf B	3	40	1734	96		15s	mo	yes
Scheduling genbuf C	3	45	1503	88		4s	mo	yes
Scheduling counter64 D	3	54	2098	108		26s	14s	yes
Scheduling counter64 E	3	37	1454	83		16s	19s	yes
Scheduling counter64 F	3	19	21391	19	19	89s	0s	no
Planning genbuf A	2	2	17	4		6s	mo	yes
Planning genbuf B	2	2	24	5		9s	mo	yes
Planning genbuf C	2	9	1156	5	5	266s	mo	no
Planning stateless D	2	3	50	9		2s	22s	yes
Planning stateless E	2	2	17	4		2s	4s	yes
Planning stateless F	2	8	973	5	5	10s	2s	no

Membership, inclusion, and containment queries are answered by TChecker. For the synthesis oracle, we used the game solver [Abssynthe](#) [41]. Abssynthe’s input format is the and-inverter graphs format (AIG). For translating Verilog modules to AIG circuits, we use [berkeley-abc](#) and [yosys](#). Abssynthe is able to compute the winning strategy $\mathfrak{S}$ for the winning player; it also computes the system controlled by $\mathfrak{S}$ in this case as an AIG circuit. The strategy containment oracle is implemented using NuSMV; since $\underline{H}$ is deterministic, one can complement it, and check whether the intersection with $(\mathcal{G} \parallel \overline{H})^\mathfrak{S}$ is empty.

The tool uses two Java threads to implement both learning phases, which are interrupted and continued when switching phases. Note that the very first learning

step of $\overline{H}$ and $\underline{H}$ can be parallelized since the first underapproximation conjecture $\underline{H}$ does not depend on $\mathfrak{S}$.

We evaluate our algorithm with two classes of benchmarks. The only tool to which we compare is Uppaal-TIGA [42] since Synthia [20] is not available anymore, and we are not aware of any other timed game solver.

In the scheduling benchmarks, there are two sporadic tasks that arrive nondeterministically, but constrained by the timed automaton. The controller must schedule these using two machines which have internal states, modeled either by a simple 8-bit counter, or by a **genbuf** circuit from the SYNTCOMP database. The scheduling duration depends on the internal state: some states require executing two external tasks, some others require executing three. The external task has a nondeterministic duration constrained by the timed automaton. The internal states change when a task is finalized. The controller loses if all machines are busy upon the arrival of a new task, or if it schedules a task on a busy machine. Uppaal TIGA was able to solve the counter models since they induce a smaller state space, but failed at the genbuf models. The compositional algorithm could efficiently handle these models. Uppaal was generally able to determine very quickly when the model is not controllable by finding a small counterstrategy, while the compositional algorithm had an overhead: it had to learn $\overline{H}$ and $\underline{H}$ before it can find and check the counterexample.

In the planning benchmarks, a robot and an obstacle is moving in a 6×6 grid (or 9×9 for the stateless case). Each agent can decide to move to an adjacent cell when they are scheduled, and the scheduling times are determined by a timed automaton. The goal of the robot is to avoid the obstacles. In the genbuf case, there are moreover internal states that can cause a glitch and prevent the agents from performing their moves, depending on their states. Uppaal TIGA was not able to manage the large state space unlike the compositional algorithm in this case, but both were able to solve the stateless case.

3.5 Extensions and Limitations

3.5.1 Timed Automata Decomposition vs. Extrapolation

Lemma 2 shows that one can consider timed automata of the form $\mathcal{A}||\mathcal{T}$ without loss of generality; and gives an effective transformation to obtain this form from any given timed automaton. The proof presents a transformation in which $\mathcal{T}'$ has a single location. While this is a theoretical transformation, it is possible to add more locations in $\mathcal{T}'$ to make the model more readable. One example is Fig. 1 where the timed automaton is given as the product of two timed automata, each with two locations.

Our setting is defined for “small” timed automata $\mathcal{T}'$ on which model checking queries must be fast because the compositional algorithm makes a large number of such queries (see Table 1); and we saw in Section 2.4 that each such query was indeed fast in our experiments (10-100ms).

Nevertheless, in the future, we would like to target models where $\mathcal{T}'$ can also be large; this could be possible with improved and faster learning algorithms which require less membership queries or use parallelization [43]. When $\mathcal{T}'$ is large, the way $\mathcal{T}'$ is modelled might have an impact on the performance of timed automata model

checking. In fact, our algorithm uses zone-based model checkers which rely on sound and complete abstractions of the state space, the so-called *extrapolation operators* on zones [2], including more precise abstraction operators that are location-dependent [44]. We briefly sketch here the idea behind these operators, and how transforming a given timed automaton as above might impact verification performance.

Zone-based verification of timed automata ([6, 45]) relies on the representation of sets of clock valuations using particular polyhedra called *zones* while discrete locations are enumerated explicitly. To guarantee termination, one must ensure to have a finite number of zones. This is achieved using extrapolation operators whose idea is the following: given a zone, any upper bound $x < k$ can be removed if k is larger than the largest constant that appears in the timed automaton. In fact, intuitively, once x is above the largest constant, then its value does not matter. A similar operation is applied to replace each lower bound $x > k$ by $x > M$ where M is the largest constant, provided that $k > M$. This ensures that only a finite number of zones is dealt with during state space exploration. There have been several improvements over these ideas, including distinguishing maximal constants that appear in lower bounds and upper bounds [2], and location dependent maximal constants [44]. In the latter optimization, the maximal constant for clock x at a given location ℓ is computed among those that are reachable from ℓ until a reset on x is seen. For instance, while 100 is the maximal constant with which y is compared in Fig. 2, at ℓ_0 , only an edge with a $y \geq 1$ is reachable; thus the *local* maximal constant is 1. This greatly improves performance in some cases.

While the transformation of Lemma 2 preserves the possibility of using extrapolation w.r.t. maximal constants for each clock, the information on *locality* disappears, that is, we can no longer use smaller maximal constants in some parts of the state space. There might be a tradeoff to be found between ensuring having a small number locations in $\mathcal{T}'$ and having enough structure so that extrapolation is efficient. While a large number of locations in $\mathcal{T}'$ can render timed automata model checkers, but also finite automata learning inefficient, having too few locations might mean that some optimized extrapolations will not be used. Tuning such an incremental approach is not obvious: the performance of extrapolation is difficult to predict, and adding more locations does not automatically accelerate model checking queries (location-based constants do not always help improve performance) but it will slow down the learning process.

We leave the development of such decomposition schemes and their empirical evaluations along with new learning algorithms for future work.

3.5.2 Compositional Verification of Networks of Timed Automata

This approach could also be applied to networks of timed automata, that is, parallel compositions of two timed automata. We indeed have the following lemma.

Lemma 6. *Consider timed automata $\mathcal{T}_i = (L_i, \ell_0^i, \Sigma_i, \text{Inv}_i, \mathcal{C}_i, E_i, F_i)$ for $i \in \{1, 2\}$, and two finite automata H_i , each on alphabet Σ_i , such that $\mathcal{L}(\mathcal{T}_i) \subseteq \mathcal{L}(H_i)$. We have $\mathcal{L}(\mathcal{T}_1 \parallel \mathcal{T}_2) \subseteq \mathcal{L}(H_1 \parallel H_2)$.*

Proof. Consider a trace $w = \sigma_1 \dots \sigma_n \in \mathcal{L}(\mathcal{T}_1 \parallel \mathcal{T}_2)$, and an accepting run

$$((\ell_1, \ell'_1), v_1) e_1 ((\ell_2, \ell'_2), v_2) e_2 \dots e_{m-1} ((\ell_m, \ell'_m), v_m)$$

over this trace. Let w^i be the projection of w to Σ_i . By definition of the parallel composition, if we remove edges with label in $\Sigma_i \setminus \Sigma_{3-i}$ from the above run, this yields a run in $\mathcal{T}_i$ over w^i . By $\mathcal{L}(\mathcal{T}_i) \subseteq \mathcal{L}(H_i)$, there exists an accepting run in H_i over trace w^i . By combining these runs, one obtains a run in $H_1 \parallel H_2$ over w . $\square$

As a consequence, one could derive an algorithm as follows. Instantiate two instances of a finite automaton learning algorithm. Each one learns an overapproximation H_i of $\mathcal{L}(\mathcal{T}_i)$. Once two hypothesis automata H_1, H_2 are obtained, we model check $H_1 \parallel H_2$: if the property holds, then we stop and conclude that it also holds in $\mathcal{T}_1 \parallel \mathcal{T}_2$ by Lemma 6. Otherwise, we check whether the obtained counterexample is realizable in both $\mathcal{T}_i$, in which case a counterexample is found. If it is not realizable in some $\mathcal{T}_i$, then we provide a negative sample to the learning algorithm for H_i , obtain a new hypothesis automaton, and repeat.

This method is sound but not complete, because we have $\mathcal{L}(\mathcal{T}_1 \parallel \mathcal{T}_2) \neq \mathcal{L}(\mathcal{T}_1) \cap \mathcal{L}(\mathcal{T}_2)$ in general. This is shown in the next example.



Fig. 7: A network of timed automata with two components $\mathcal{T}_1$ (on the left) over alphabet $\{a\}$, and $\mathcal{T}_2$ (on the right) over alphabet $\{b\}$.

Example 2. Consider the network of timed automata $\mathcal{T}_1 \parallel \mathcal{T}_2$ given in Fig. 7, and write $\Sigma = \{a, b\}$. We have $aa\Sigma^* \notin \mathcal{L}(\mathcal{T}_1 \parallel \mathcal{T}_2)$ since the clocks make sure two a 's cannot be repeated. However, any DFA that overapproximate $\mathcal{L}(\mathcal{T}_1)$ must accept the language a^* since $\mathcal{L}(\mathcal{T}_1) = a^*$; and any overapproximation of $\mathcal{L}(\mathcal{T}_2)$ accepts b^* . For such overapproximations H_1, H_2 , we obtain $(a + b)^* = \mathcal{L}(H_1 \parallel H_2)$, which fails to exclude factors aa .

Despite incompleteness, we believe this method can be useful in practice. We leave the identification of useful cases and an empirical evaluation for future work.

3.5.3 The Chinese Remainders Issue

On some particular models, building finite automata abstractions for the timed automaton component $\mathcal{T}$ may not be a good idea. In fact, clocks can be used to build systems that are equivalent to exponentially large finite automata [46]. We sketch here one such case that appears in [46]. Consider the network of timed automaton $\mathcal{T}$ that is the parallel composition of k self-loops, each enforcing a distinct event exactly every p_i time units, as shown in Fig. 8. Intuitively, by the Chinese remainders theorem, all discrete clock valuations reached from time 0 to $p_1 p_2 \dots p_k - 1$ recognize different languages (when seen as initial states). Therefore, any finite automaton that recognizes the same language as the network described in Fig. 8 is of size exponential in k .

Although our algorithm only builds an overapproximation, algorithms such as L^* does spend a lot of time before even making the first equivalence query (which is an inclusion query in our case). In practice, when such sequences of prime numbers and

equality constraints are avoided, often smaller finite-state abstractions were obtained. A theoretical study of the size of finite automata approximations of such timed automata as a function of the constants that appear in the description is the object of future work.

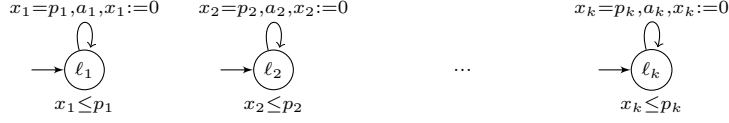


Fig. 8: A network of timed automata with k components, and $p_1 < \dots < p_k$ prime numbers.

4 Conclusion

Related Works

Perhaps the most closely related approach to our compositional model checking algorithm is trace abstraction refinement [47]. This was originally applied to program verification, and consists in building a network of finite automata that recognizes the program’s control flow paths that are infeasible. One refines this language by model checking the control flow graph intersected with the complement of the automaton. Thus, the semantics of the variables of the program are abstractly represented by the finite automaton. This idea was applied to timed automata as well [48, 49]. However, the generalization of the counterexamples which ensures convergence turns out to be less effective in timed automata. We attempted at obtaining an implementation, but could only confirm the poor performance for model checking timed automata as in [49] (we do not include these results here). It might be that simpler graph structures such as control flow graphs of programs are necessary for this approach to scale; further investigation is also necessary to study better generalization methods.

The learning-based compositional reasoning approach of [23] is also related to counter-example guided abstraction refinement (CEGAR) [50]. In fact, the automata learning algorithm builds an overapproximation of one of the components, and refines it as needed, guided by counterexamples. The difference is that, instead of using predicates, one uses automata to represent the overapproximation. A discussion can also be found in [23].

Learning algorithms for event-recording automata, a subset of timed automata were studied in [51]. The algorithm of [23] was extended for these automata in [30]. In the context of parameter synthesis with learning, parameterized systems were seen as a parallel composition of a non-parameterized component, and a parameterized component in [52].

Other approaches targeting the formal verification of real-time systems with large discrete state spaces include encodings of timed automata semantics in Boolean logic include [53, 54]. An extension of and-inverter graphs were used in [55] that uses predicates to represent the state space of linear hybrid automata.

The abstract interpretation of games were studied in [56] that presents a theory allowing one to define under- and over-approximations. Abstraction-refinement algorithms based on counterexamples were given in [57, 58]. These ideas were applied to timed games in [20]. Several abstraction-refinement and compositional algorithms were given in [41, 59] for solving finite-state games given as Boolean circuits. The [synthesis competition](#) gathers every year researchers who present their game solvers [21, 60].

Perspectives

The algorithm we presented builds finite-state abstractions of real-time constraints, that it represents as DFA. The approach is well adapted when the interaction alphabet between $\mathcal{A}$ and $\mathcal{T}$ is small; this is the case, for instance, for distributed systems where the time constraints are used to describe the approximate period with which each process communicates with its neighbors; so the alphabet contains only a few symbols per process. Some of the benchmarks we considered are models of such systems. The approach is less convenient for time-intensive systems such as, say, job shop scheduling problems where a separate alphabet symbol is needed for each task.

As future work, we would like to understand when various abstraction schemes are efficient among the approach presented here, the predicate-abstraction approach, and zone-based state-space exploration. Currently, all algorithms fail in some benchmarks. Understanding the strengths of each algorithm might help designing a uniformly better solution. Currently, we can only verify linear properties; one might verify branching-time properties by learning automata with a stronger notion of equivalence such as bisimulation. In fact, an important limitation is due to learning being slow for large alphabets. Our setting could be extended to deal with large or symbolic alphabets *e.g.* [61, 62].

For synthesis, our setting is currently restricted by the abstractions we use since when the algorithm rejects the instance, we cannot conclude whether the system is controllable or not. Using both the under- and overapproximations within the finite-state synthesis, for instance, using the three-valued abstraction approach [58] might allow us to render the approach complete, and to consider a larger class of timed games such as those that allow Controller to select nonzero delays.

The treatment of ω -regular traces is significantly more difficult and requires different assume-guarantee rules; this direction is explored in [63].

Acknowledgement

We thank Frédéric Herbreteau who suggested the content of Section 3.5.1. The work was partially supported by grants from *Agence Nationale de la Recherche (ANR)* projects ANR-22-CE48-0012 Bisous, and ANR-23-CE48-0005 Pavedys.

This preprint has not undergone peer review or any post-submission improvements or corrections. The Version of Record of this article is published in Journal of Automated Reasoning, and is available online at <https://doi.org/10.1007/s10817-025-09730-z>.

Data Availability

Source codes, executables, and benchmark data are available as an artifact [64].

References

- [1] Alur, R., Dill, D.L.: A theory of timed automata. *Theoretical Computer Science* **126**(2), 183–235 (1994)
- [2] Behrmann, G., Bouyer, P., Larsen, K.G., Pelanek, R.: Lower and upper bounds in zone-based abstractions of timed automata. *Int. J. Softw. Tools Technol. Transf.* **8**(3), 204–215 (2006) <https://doi.org/10.1007/s10009-005-0190-0>
- [3] Behrmann, G., David, A., Larsen, K.G., Håkansson, J., Pettersson, P., Yi, W., Hendriks, M.: UPPAAL 4.0. In: *Third International Conference on the Quantitative Evaluation of Systems (QEST 2006)*, 11–14 September 2006, Riverside, California, USA, pp. 125–126 (2006). <https://doi.org/10.1109/QEST.2006.59>
- [4] Herbreteau, F., Point, G.: The TChecker tool and librairies. <https://github.com/ticktac-project/tchecker>
- [5] Sun, J., Liu, Y., Dong, J.S., Pang, J.: Pat: Towards flexible verification under fairness. In: *Proceedings of the 21th International Conference on Computer Aided Verification (CAV’09)*. *Lecture Notes in Computer Science*, vol. 5643, pp. 709–714. Springer, Berlin Heidelberg (2009)
- [6] Bouyer, P., Gastin, P., Herbreteau, F., Sankur, O., Srivathsan, B.: Zone-based verification of timed automata: extrapolations, simulations and what next? In: *20th International Conference on Formal Modeling and Analysis of Timed Systems (FORMATS 2022)*. Springer, Berlin Heidelberg (2022). <https://doi.org/10.48550/arXiv.2207.07479> . <https://doi.org/10.48550/arXiv.2207.07479>
- [7] Wang, F.: Symbolic verification of complex real-time systems with clock-restriction diagram. In: *Proc. 21st International Conference on Formal Techniques for Networked and Distributed Systems (FORTE’01)*. *IFIP Conference Proceedings*, vol. 197, pp. 235–250. Kluwer, ??? (2001)
- [8] Beyer, D., Lewerentz, C., Noack, A.: Rabbit: A tool for BDD-based verification of real-time systems. In: *Proc. 15th International Conference on Computer Aided Verification (CAV’03)*. *Lecture Notes in Computer Science*, vol. 2725, pp. 122–125. Springer, ??? (2003)
- [9] Ehlers, R., Fass, D., Gerke, M., Peter, H.-J.: Fully symbolic timed model checking using constraint matrix diagrams. In: *Proc. 31th IEEE Real-Time Systems Symposium (RTSS’10)*, pp. 360–371. IEEE Computer Society Press, ??? (2010). <https://doi.org/10.1109/RTSS.2010.36>
- [10] Nguyen, T.K., Sun, J., Liu, Y., Dong, J.S., Liu, Y.: Improved BDD-based discrete analysis of timed systems. In: *Proc. 20th International Symposium on Formal Methods (FM’12)*, vol. 7436, pp. 326–340. Springer, ??? (2012)

- [11] Thierry-Mieg, Y.: Symbolic model-checking using ITS-tools. In: Proc. 21st International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS'15), pp. 231–237 (2015). Springer
- [12] Cimatti, A., Griggio, A., Mover, S., Tonetta, S.: IC3 modulo theories via implicit predicate abstraction. In: Proc. 20th International Conference on Tools and Algorithms for the Construction and Analysis of Systems (TACAS'14). Lecture Notes in Computer Science, vol. 8413, pp. 46–61 (2014)
- [13] Cimatti, A., Griggio, A., Magnago, E., Roveri, M., Tonetta, S.: Extending nuxmv with timed transition systems and timed temporal properties. In: International Conference on Computer Aided Verification, pp. 376–386 (2019). Springer
- [14] Roussanaly, V., Sankur, O., Markey, N.: Abstraction refinement algorithms for timed automata. In: Dillig, I., Tasiran, S. (eds.) Computer Aided Verification (CAV'19), pp. 22–40. Springer, Cham (2019). https://link.springer.com/chapter/10.1007%2F978-3-030-25540-4_2
- [15] Thomas, W.: On the synthesis of strategies in infinite games. In: Mayr, E.W., Puech, C. (eds.) STACS 95, pp. 1–13. Springer, Berlin, Heidelberg (1995)
- [16] Maler, O., Pnueli, A., Sifakis, J.: On the synthesis of discrete controllers for timed systems (an extended abstract). In: STACS, pp. 229–242 (1995)
- [17] Asarin, E., Maler, O., Pnueli, A.: Symbolic controller synthesis for discrete and timed systems. In: Hybrid Systems II. LNCS, vol. 999, pp. 1–20. Springer, ??? (1995)
- [18] Cassez, F., David, A., Fleury, E., Larsen, K.G., Lime, D.: Efficient on-the-fly algorithms for the analysis of timed games. In: Proc. 16th International Conference on Concurrency Theory (CONCUR'05). Lecture Notes in Computer Science, vol. 3653, pp. 66–80. Springer, ??? (2005)
- [19] Behrmann, G., Cougnard, A., David, A., Fleury, E., Larsen, K.G., Lime, D.: UPPAAL-Tiga: Time for playing games! In: Proc. 19th International Conference on Computer Aided Verification (CAV'07). Lecture Notes in Computer Science, vol. 4590, pp. 121–125. Springer, ??? (2007)
- [20] Peter, H.-J., Ehlers, R., Mattmüller, R.: Synthia: Verification and synthesis for timed automata. In: International Conference on Computer Aided Verification, pp. 649–655 (2011). Springer
- [21] Jacobs, S., Bloem, R., Brenguier, R., Ehlers, R., Hell, T., Könighofer, R., Pérez, G.A., Raskin, J.-F., Ryzhyk, L., Sankur, O., *et al.*: The first reactive synthesis competition (syntcomp 2014). International journal on software tools for technology transfer **19**(3), 367–390 (2017)

- [22] Bertin, V., Closse, E., Poize, M., Pulou, J., Sifakis, J., Venier, P., Weil, D., Yovine, S.: Taxys=esterel+kronos. a tool for verifying real-time properties of embedded systems. In: Proceedings of the 40th IEEE Conference on Decision and Control (Cat. No.01CH37228), vol. 3, pp. 2875–28803 (2001). <https://doi.org/10.1109/CDC.2001.980712>
- [23] Păsăreanu, C.S., Giannakopoulou, D., Bobaru, M.G., Cobleigh, J.M., Barringer, H.: Learning to divide and conquer: applying the L* algorithm to automate assume-guarantee reasoning. *Formal Methods in System Design* **32**(3), 175–205 (2008)
- [24] Angluin, D.: Learning regular sets from queries and counterexamples. *Information and Computation* **75**(2), 87–106 (1987) [https://doi.org/10.1016/0890-5401\(87\)90052-6](https://doi.org/10.1016/0890-5401(87)90052-6)
- [25] Isberner, M., Howar, F., Steffen, B.: The ttt algorithm: a redundancy-free approach to active automata learning. In: International Conference on Runtime Verification, pp. 307–322 (2014). Springer
- [26] Aceto, L., Bouyer, P., Burgueño, A., Larsen, K.G.: The power of reachability testing for timed automata. *Theoretical Computer Science* **300**(1), 411–475 (2003) [https://doi.org/10.1016/S0304-3975\(02\)00334-1](https://doi.org/10.1016/S0304-3975(02)00334-1)
- [27] Sankur, O.: Timed Automata Verification and Synthesis via Finite Automata Learning. In: TACAS 2023 - 29th International Conference on Tools and Algorithms for the Construction and Analysis of Systems, Paris, France (2023). <https://hal.science/hal-03947462>
- [28] Rivest, R.L., Schapire, R.E.: Inference of finite automata using homing sequences. *Information and Computation* **103**(2), 299–347 (1993) <https://doi.org/10.1006/inco.1993.1021>
- [29] Clarke, E.M., Henzinger, T.A., Veith, H., Bloem, R., *et al.*: Handbook of Model Checking vol. 10. Springer, ??? (2018)
- [30] Lin, S.-W., André, É., Liu, Y., Sun, J., Dong, J.S.: Learning assumptions for compositional verification of timed systems. *Transactions on Software Engineering* **40**(2), 137–153 (2014) <https://doi.org/10.1109/TSE.2013.57>
- [31] Isberner, M., Howar, F., Steffen, B.: The open-source learnlib. In: Kroening, D., Păsăreanu, C.S. (eds.) *Computer Aided Verification*, pp. 487–495. Springer, Cham (2015)
- [32] Cimatti, A., Griggio, A.: Software model checking via ic3. In: Madhusudan, P., Seshia, S.A. (eds.) *Computer Aided Verification*, pp. 277–293. Springer, Berlin, Heidelberg (2012)

- [33] Delporte-Gallet, C., Devismes, S., Fauconnier, H.: Robust stabilizing leader election. In: Masuzawa, T., Tixeuil, S. (eds.) *Stabilization, Safety, and Security of Distributed Systems*, pp. 219–233. Springer, Berlin, Heidelberg (2007)
- [34] Maróti, M., Kusy, B., Simon, G., Lédeczi, A.: The flooding time synchronization protocol. In: *Proceedings of the 2Nd International Conference on Embedded Networked Sensor Systems. SenSys '04*, pp. 39–49. ACM, New York, NY, USA (2004). <https://doi.org/10.1145/1031495.1031501> . <http://doi.acm.org/10.1145/1031495.1031501>
- [35] McInnes, A.I.: Model-checking the flooding time synchronization protocol. In: *Control and Automation, 2009. ICCA 2009. IEEE International Conference On*, pp. 422–429 (2009). <https://doi.org/10.1109/ICCA.2009.5410508>
- [36] Kusy, B., Abdelwahed, S.: Ftsp protocol verification using spin (2006)
- [37] Sankur, O., Talpin, J.: An abstraction technique for parameterized model checking of leader election protocols: Application to FTSP. In: *Tools and Algorithms for the Construction and Analysis of Systems - 23rd International Conference, TACAS 2017, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2017, Uppsala, Sweden, April 22–29, 2017, Proceedings, Part I*, pp. 23–40 (2017). https://doi.org/10.1007/978-3-662-54577-5_2 . https://doi.org/10.1007/978-3-662-54577-5_2
- [38] Dierks, H.: Time, Abstraction and Heuristics - Automatic Verification and Planning of Timed Systems Using Abstraction and Heuristics. *Berichte aus dem Department für Informatik / Universität Oldenburg / Fachbereich Informatik*, vol. 01-06 (2006)
- [39] Bengtsson, J., Yi, W.: Timed automata: Semantics, algorithms and tools. In: Desel, J., Reisig, W., Rozenberg, G. (eds.) *Lectures on Concurrency and Petri Nets. Lecture Notes in Computer Science*, vol. 2098, pp. 87–124. Springer, ??? (2004)
- [40] Luo, G., Bochmann, G., Petrenko, A.: Test selection based on communicating nondeterministic finite-state machines using a generalized wp-method. *IEEE Transactions on Software Engineering* **20**(2), 149–162 (1994) <https://doi.org/10.1109/32.265636>
- [41] Brenguier, R., Pérez, G.A., Raskin, J.-F., Sankur, O.: Abssynthe: abstract synthesis from succinct safety specifications. In: Chatterjee, K., Ehlers, R., Jha, S. (eds.) *Proceedings 3rd Workshop On Synthesis (SYNT'14). Electronic Proceedings in Theoretical Computer Science*, vol. 157, pp. 100–116. Open Publishing Association, ??? (2014). <https://doi.org/10.4204/EPTCS.157.11> . <http://arxiv.org/abs/1407.5961v1>
- [42] Behrmann, G., Cougnard, A., David, A., Fleury, E., Larsen, K.G., Lime, D.: Uppaal-tiga: Time for playing games! In: *International Conference on Computer Aided Verification*, pp. 121–125 (2007). Springer

- [43] Valizadeh, M., Fijalkow, N., Berger, M.: Ltl learning on gpus. In: International Conference on Computer Aided Verification, pp. 209–231 (2024). Springer
- [44] Behrmann, G., Bouyer, P., Fleury, E., Larsen, K.G.: Static guard analysis in timed automata verification. In: Garavel, H., Hatcliff, J. (eds.) Proceedings of the 9th International Conference on Tools and Algorithms for Construction and Analysis of Systems (TACAS’03). Lecture Notes in Computer Science, vol. 2619, pp. 254–277. Springer, Warsaw, Poland (2003). <http://www.lsv.ens-cachan.fr/Publis/PAPERS/PDF/BBFL-tacas-2003.pdf>
- [45] Henzinger, T.A., Nicollin, X., Sifakis, J., Yovine, S.: Symbolic model checking for real time systems. *Information & Computation* **111**(2), 193–244 (1994)
- [46] Brenguier, R., Göller, S., Sankur, O.: A comparison of succinctly represented finite-state systems. In: Koutny, M., Ulidowski, I. (eds.) Proceedings of the 23rd International Conference on Concurrency Theory (CONCUR’12). Lecture Notes in Computer Science, vol. 7454, pp. 147–161. Springer, Newcastle, UK (2012). https://doi.org/10.1007/978-3-642-32940-1_12 . <http://www.lsv.ens-cachan.fr/Publis/PAPERS/PDF/BGS-concur12.pdf>
- [47] Heizmann, M., Hoenicke, J., Podelski, A.: Refinement of trace abstraction. In: International Static Analysis Symposium, pp. 69–85 (2009). Springer
- [48] Wang, W., Jiao, L.: Trace abstraction refinement for timed automata. In: Cassez, F., Raskin, J.-F. (eds.) Automated Technology for Verification and Analysis, pp. 396–410. Springer, Cham (2014)
- [49] Cassez, F., Jensen, P.G., Larsen, K.G.: Verification and parameter synthesis for real-time programs using refinement of trace abstraction. *Fundam. Informaticae* **178**(1-2), 31–57 (2021) <https://doi.org/10.3233/FI-2021-1997>
- [50] Clarke, E., Grumberg, O., Jha, S., Lu, Y., Veith, H.: Counterexample-guided abstraction refinement for symbolic model checking. *Journal of the ACM (JACM)* **50**(5), 752–794 (2003)
- [51] Grinchtein, O., Jonsson, B., Leucker, M.: Learning of event-recording automata. *Theoretical Computer Science* **411**(47), 4029–4054 (2010) <https://doi.org/10.1016/j.tcs.2010.07.008>
- [52] André, É., Lin, S.-W.: Learning-based compositional parameter synthesis for event-recording automata. In: Bouajjani, A., Silva, A. (eds.) Formal Techniques for Distributed Objects, Components, and Systems, pp. 17–32. Springer, Cham (2017)
- [53] Kindermann, R., Junttila, T., Niemela, I.: Modeling for symbolic analysis of safety instrumented systems with clocks. In: Proc. 11th International Conference on Application of Concurrency to System Design (ACSD’11), pp. 185–194. IEEE Computer Society Press, ??? (2011). <https://doi.org/10.1109/ACSD.2011.29>

- [54] Seshia, S.A., Bryant, R.E.: Unbounded, fully symbolic model checking of timed automata using boolean methods. In: Hunt, W.A., Somenzi, F. (eds.) *Computer Aided Verification*, pp. 154–166. Springer, Berlin, Heidelberg (2003)
- [55] Damm, W., Dierks, H., Disch, S., Hagemann, W., Pigorsch, F., Scholl, C., Waldmann, U., Wirtz, B.: Exact and fully symbolic verification of linear hybrid automata with large discrete state spaces. *Science of Computer Programming* **77**(10), 1122–1150 (2012) <https://doi.org/10.1016/j.scico.2011.07.006>
- [56] Henzinger, T.A., Majumdar, R., Mang, F., Raskin, J.-F.: Abstract interpretation of game properties. In: Palsberg, J. (ed.) *Static Analysis*, pp. 220–239. Springer, Berlin, Heidelberg (2000)
- [57] Henzinger, T.A., Jhala, R., Majumdar, R.: Counterexample-guided control. In: *International Colloquium on Automata, Languages, and Programming*, pp. 886–902 (2003). Springer
- [58] de Alfaro, L., Roy, P.: Solving games via three-valued abstraction refinement. *Information and Computation* **208**(6), 666–676 (2010) <https://doi.org/10.1016/j.ic.2009.05.007> . Special Issue: 18th International Conference on Concurrency Theory (CONCUR 2007)
- [59] Brenguier, R., Pérez, G.A., Raskin, J.-F., Sankur, O.: Compositional algorithms for succinct safety games. In: Černý, P., Kuncak, V., Parthasarathy, M. (eds.) *Proceedings Fourth Workshop on Synthesis (SYNT’15)*, San Francisco, CA, USA, 18th July 2015. *Electronic Proceedings in Theoretical Computer Science*, vol. 202, pp. 98–111. Open Publishing Association, ??? (2016). <https://doi.org/10.4204/EPTCS.202.7>
- [60] Jacobs, S., Perez, G.A., Abraham, R., Bruyere, V., Cadilhac, M., Colange, M., Delfosse, C., Dijk, T., Duret-Lutz, A., Faymonville, P., et al.: The reactive synthesis competition (syntcomp): 2018-2021. arXiv preprint arXiv:2206.00251 (2022)
- [61] Maler, O., Mens, I.-E.: Learning regular languages over large alphabets. In: Ábrahám, E., Havelund, K. (eds.) *Tools and Algorithms for the Construction and Analysis of Systems*, pp. 485–499. Springer, Berlin, Heidelberg (2014)
- [62] Maler, O., Mens, I.-E.: A generic algorithm for learning symbolic automata from membership queries. In: Aceto, L., Bacci, G., Bacci, G., Ingólfssdóttir, A., Legay, A., Mardare, R. (eds.) *Models, Algorithms, Logics and Tools: Essays Dedicated to Kim Guldstrand Larsen on the Occasion of His 60th Birthday*, pp. 146–169. Springer, Cham (2017). https://doi.org/10.1007/978-3-319-63121-9_8 . https://doi.org/10.1007/978-3-319-63121-9_8
- [63] Sankur, O.: Automatic assume-guarantee reasoning for safety and liveness using passive learning. Preprint (2024) <https://doi.org/10.21203/rs.3.rs-3910982/v1>

- [64] Sankur, O.: Artifact for the paper: Timed Automata Verification and Synthesis via Finite Automata Learning. Zenodo (2022). <https://doi.org/10.5281/zenodo.7487508>